

广播协议

故障：fautes logicielles (de conception ou de programmation)

fautes matérielles (ou plus généralement système)

piratage 黑客攻击

受影响的组件：进程、处理器、通信通道

故障分类：faute franche 直接（停止响应/传输）

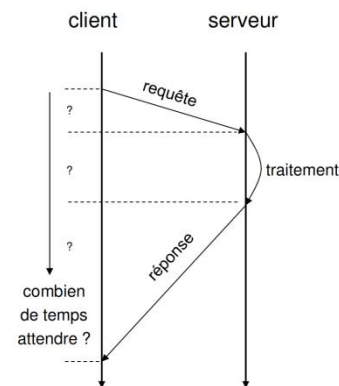
faute d'omission 遗漏（暂时无法发送）

faute temporelle 时间（过早/过晚）

faute byzantine 拜占庭（同时包含以上多种）

复杂度从上到下依次增大

时间模型



1. 同步系统 (Synchronous system)

- 消息传输延迟有已知上界。
- 进程速度差异有上界。
- 举例：理论上的“锁步执行”系统。
- 在这种系统里，可以用超时来完美检测故障。

2. 异步系统 (Asynchronous system)

- 消息延迟没有上界。
- 进程速度没有上界。
- 现实网络接近这种模型。
- 问题：在异步系统里，你永远不知道一个进程是真的挂了还是“只是慢”。
- 结果：出现了 FLP 不可能性定理——在异步+crash fault 模型下，确定性 consensus 不可能实现。

3. 部分同步系统 (Partially synchronous system)

- 介于两者之间：消息延迟/进程速度有上界，但上界值未知。
- 更贴近实际，很多算法（比如 Paxos、Raft）都基于这个模型。

★ 直观理解：

- 同步模型：像坐公交，最多等 5 分钟车一定来。
- 异步模型：像等朋友，不知道是堵车了还是放你鸽子了。
- 部分同步：你知道 TA 最多两个小时会到，但不知道具体几分钟。

同步系统：DBC 已知有界时间/偏差模型

部分同步系统：DBI 未知有界时间/偏差模型

异步系统：DBN 无界时间/偏差模型

FLP: 在异步系统中，即使存在明显的错误，也不能确定性地解决共识问题。

共识的正式规范：

-终止 terminaison：任何正确的流程最终都会做出决定

-协议 accord：两个流程不能做出不同的决定

-完整性 intégrité：一个流程最多做出一次决定

-有效性 validité：如果 v 是决定的值，则 v 是 v_i 之一

我们的模型：

- ① 一组独立的顺序进程
每个进程一次只执行一个动作
- ② 通过信息交换进行通信
没有共享内存
进程的输入是接收的信息，输出是发送的信息
- ③ 异步系统（通常被认为是）：
异步通信
不假设消息路由时间（无上限）
异步处理
没有关于进程演变的时间假设
- ④ 没有共同时钟

分布式系统中各个进程必须协作完成任务。

在分布式系统中，为了保证数据一致性，更新需要同步到所有副本上

并行计算应用 *Application de calcul parallèle*：如果消息丢失，则停止计算。

复制服务器 *Serveur répliqué*：如果在所有进程上接收消息的顺序不同，则它们是不一致的。

广播定义：

发送进程向一组进程发送信息。

组：一组进程（群成员），通过广播而非点对点传输寻址。

broadcast (m) 消息 **m** 向进程 **p** 广播

deliver (m) 进程 **p** 接受消息 **m**

广播是在现有的通信系统上进行的

非原子（无法保证一次广播所有进程都能接收到消息）

broadcast (m) 依赖底层的 **send(m,p)** 点对点发送

消息不会直接被应用进程^{交付}接收。

接收到的消息先通过 **recv(m)** 操作存入一个队列（“file d'attente”），然后再通过 **deliver(m)** 将消息传递给应用层进行处理。

（确保消息在接收之后有序、正确地被应用层使用）

复制透明度：消息相同，但是接受者数量和身份保密

封闭组：**broadcast (m)** 只能由组内进程调用（只能组内广播）

开放组：**broadcast (m)** 由组外进程调用（可以由外部进程向组内广播）

静态组：组的成员固定

动态组：组的成员可以自愿加入退出

问题:

程序可能崩溃 (特别是发送多条消息时)

时序无法保证

待解决:

保证广播属性 :

保证信息的传送

保证接收信息的调度

如果一个进程在整个给定的执行过程中没有崩溃, 它就被认为是正确的, 否则它就被认为是错误的。

最终接收(eventually receives): 如果一个正确的进程 p 向另一个正确的进程 q 发送消息 m , 那么 q 最终会接收到这个消息。

(可靠通道保证消息不会丢失, 尽管可能会有延迟)

一个在整个执行过程中没有发生崩溃的进程被称为---->“correct”。

如果进程在执行过程中崩溃了, 它就被称为----->“fautif”。

崩溃故障(fail-stop): 进程在运行时可能会发生故障, 并且一旦发生故障, 进程将永久停止, 不会自动恢复或重新启动。

Garantie de remise 交付保证:

Diffusion Best-effort (Best-effort Broadcast) 尽力而为广播

尽最大努力广播确保消息传递到所有正确的进程, 如果发送者是正确的。

问题: 如果发射机出现故障, 无法保证交付。‘全部接收者’

有效性 Validité: 如果 p_1 和 p_2 正确, 则由 p_1 广播的消息 m 最终由 p_2 传送。

完整性 Intégrité: 消息 m 最多传递一次, 且仅当它已由进程广播时。

算法:

Processus P :

BestEffort_broadcast (m)

. envoyer m à tous les processus y compris p /* groupe fermé */

upon recv(m) :

BestEffort_deliver(m) /* délivrer le message */

Diffusion Fiable (Reliable Broadcast) 可靠广播

如果消息 m 的发送者是正确的，那么所有正确的接收者都传递消息 m 。

如果消息 m 的发送者有故障，则所有或没有正确的进程都会传递消息 m 。

有效性 Validité: 如果一个正确的进程广播消息 m ，那么所有正确的进程都会广播 m 。

完整性 Intégrité: 消息 m 最多向任何正确的进程广播一次，并且仅当它已由一个进程广播时。

一致性 Accord: 如果一个正确的进程传递消息 m ，那么所有正确的成员都传递 m 。

如果一个正确的进程传递广播消息 m ，那么任何正确的进程也传递 m

可能的实现: 当接收到另一个进程广播的消息时，会在传递消息之前重新广播该消息。

文件

算法:

每个广播消息 m 都唯一地加盖有:

- sender(m): 发送者的身份

Processus P :

Variable locale :

rec = $\emptyset$;

Real_broadcast (m)

estampiller m avec sender(m); 为消息加盖发送者身份

envoyer m à tous les processus y compris p 包括自己

upon recv(m) do

if $m \notin \text{rec}$ 没收到过

rec $\cup = \{m\}$ m 加入 rec

if sender(m) $\neq p$ then 发送者不是 P

envoyer m à tous les processus sauf p 再次广播，降了自己

Real_deliver(m) /* délivrer le message */ 文件

优点 :

可靠性并非基于检测发射机故障

因此，该算法适用于任何时间模型

缺点:

算法效率很低: 每次广播会产生 $n(n-1)$ 条信息

因此在实际应用中无法使用

评论:

该算法不能保证任何发送顺序

Complétude (Completeness) - 完整性:

- **Forte (强完整性):** 存在一个时刻, 从此时刻开始, 每个发生故障的进程都被所有正确的进程怀疑为有故障。
- **Faible (弱完整性):** 存在一个时刻, 从此时刻开始, 每个发生故障的进程至少被一个正确的进程怀疑为有故障。

Justesse (Accuracy) - 准确性:

- **Forte (强准确性):** 没有正确的进程被怀疑为有故障。
- **Faible (弱准确性):** 至少有一个正确的进程从未被怀疑为有故障。
- **Finement forte (最终强准确性):** 存在一个时刻, 从此时刻开始, 所有正确的进程不再被任何正确的进程怀疑。
- **Finement faible (最终弱准确性):** 存在一个时刻, 从此时刻开始, 至少有一个正确的进程不再被任何正确的进程怀疑。

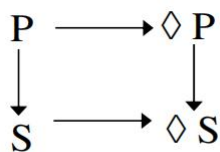
故障检测器分类

检测器分为四类, 依据其完整性和准确性:

- **P:** 强完整性 + 强准确性。
- **S:** 强完整性 + 弱准确性。
- **Q:** 弱完整性 + 强准确性。
- **W:** 弱完整性 + 弱准确性。

带菱形的表示最终 xxx

强完备性和弱完备性是等价的 (我们可以从弱完备性构建强完备性)



P 强于 S 和 $\diamond P$

$\diamond P$ 强于 $\diamond S$

S 强于 $\diamond S$

强的检测器可以推导出弱的检测器

检测器的强度反映了它对系统故障的反应和识别能力, 较强的检测器在更广泛的条件下能够更可靠、更准确地检测到故障。

问题:

对于有缺陷的流程, 不提供交付保证

Diffusion Fiable Uniforme (Uniform Reliable Broadcast) 统一可靠广播

要么全接收 (正确节点)
要么全未接收 (正确节点)

如果消息 m 是由一个进程 (错误的或正确的) 传递的, 那么任何正确的进程也以传递 m 结束。

交付

交付

统一性属性: 如果一个属性 (一致性、完整性) 适用于所有过程: 正确的和错误的, 则称该属性为统一属性。

有效性 Validité: 如果一个正确的进程广播了信息 m , 那么所有正确的进程都会传递信息 m 。

统一完整性 Intégrité uniforme: 信息 m 最多只能传递一次给任何进程 (正确的或有问题的), 而且只有当它被某个进程广播时才会被传递。

交付

交付

统一一致性 Accord uniforme: 如果一个进程 (正确的或有问题的) 传递信息 m , 那么所有正确的成员都传递信息 m 。

交付

定时可靠广播 Diffusion fiable temporisée = 可靠广播 diffusion fiable + 时间界限 borne

同步通信系统:

定时可靠广播假设系统是同步的, 这意味着消息在网络中的传输时间是有界的, 通信延迟不会无限增长。

时间界限: 存在一个常数 Δ , 使得如果某个消息 m 在时间 t 被广播, 那么在时间 $t + \Delta$ 之后, 所有正确的进程都必须已经交付消息 m 。

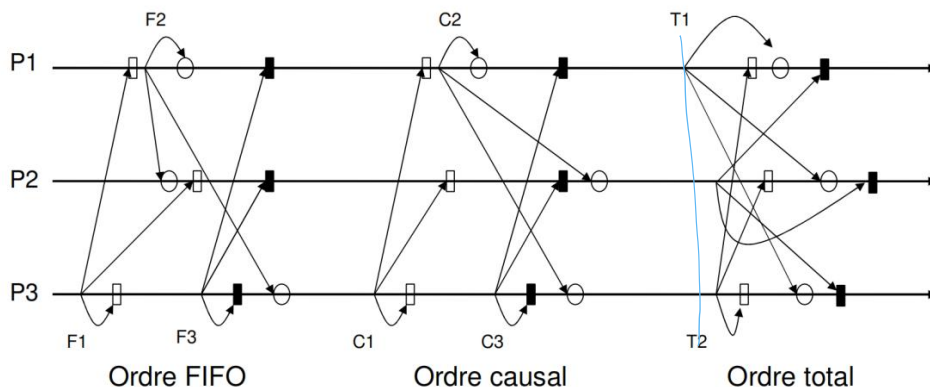
(换句话说, 消息在广播后不会无限期地等待, 必须在一个限定的时间内完成传递)

保证顺序:

全序: 消息以相同的顺序传递给所有收件人

FIFO 顺序: 如果一个成员先广播 m_1 , 然后广播 m_2 , 则任何正确的发送 m_2 的成员都会在 m_2 之前发送 m_1

因果顺序: 如果 $\text{broadcast}(m_1)$ 因果地先于 $\text{broadcast}(m_2)$, 那么任何传递 m_2 的正确进程都会在 m_2 之前传递 m_1



只在此前交付

C1 在 C2 前

与发送无关

注释:

C1 在 C2 前

所有进程交付顺序相同

因果关系广播必然是先进先出（因果关系广播可被视为先进先出顺序对组中所有进程的概括）。

FIFO 顺序和因果顺序只是部分顺序：它们对同时广播的消息的传递顺序不施加任何限制。

全序与 FIFO 顺序和因果顺序没有任何联系：它既强（信息传递的全序）又弱（广播顺序和传递顺序之间没有联系）。

全序：允许在存在多个 writer 的情况下保持服务器副本的一致性。

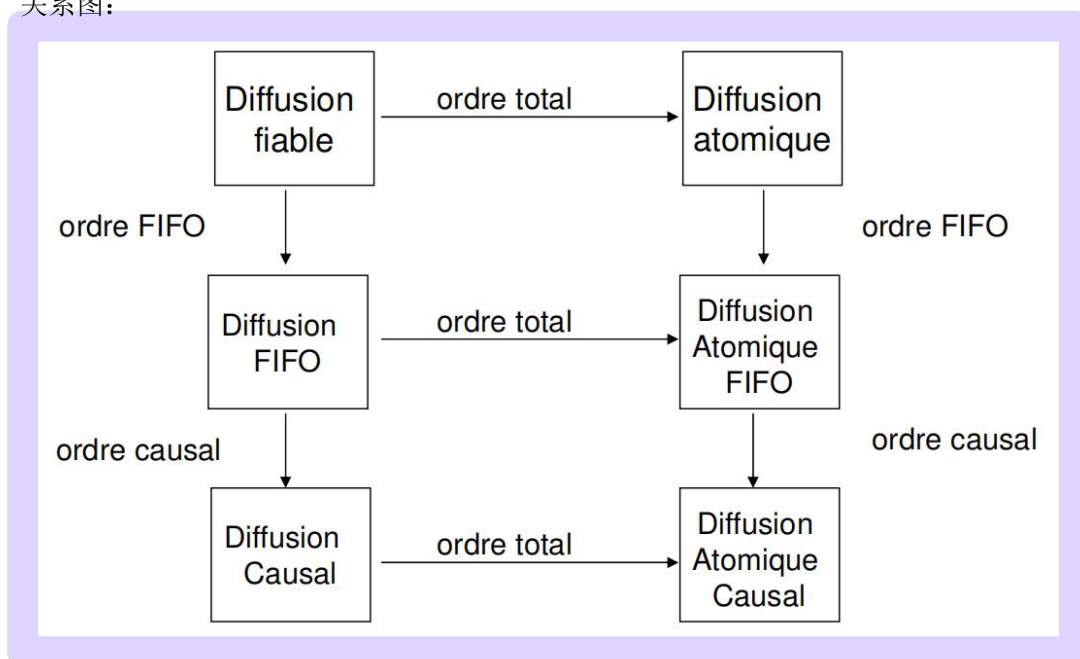
因果顺序：允许以低成本维护逻辑相关的操作序列。

FIFO 顺序：允许在存在单个写入器的情况下保持服务器副本的一致性。

维护成本：

全序 > 因果 > FIFO

关系图：



先进先出广播 Diffusion FIFO = 可靠广播 Diffusion fiable + 先进先出顺序 Ordre FIFO

因果扩散 Diffusion Causal (CBCAST) = 可靠扩散 Diffusion fiable + 因果顺序 Ordre Causal

原子扩散 Diffusion Atomique (ABCAST) = 可靠扩散 Diffusion fiable + 全序 Ordre Total

原子扩散先进先出 Diffusion Atomique FIFO = 先进先出扩散 Diffusion FIFO + 全序 Ordre Total

因果原子扩散 Diffusion Atomique Causal = 因果扩散 Diffusion Causal + 全序 Ordre Total

在可靠广播中，没有关于消息传递顺序的规范。

Diffusion FIFO = diffusion fiable + ordre FIFO

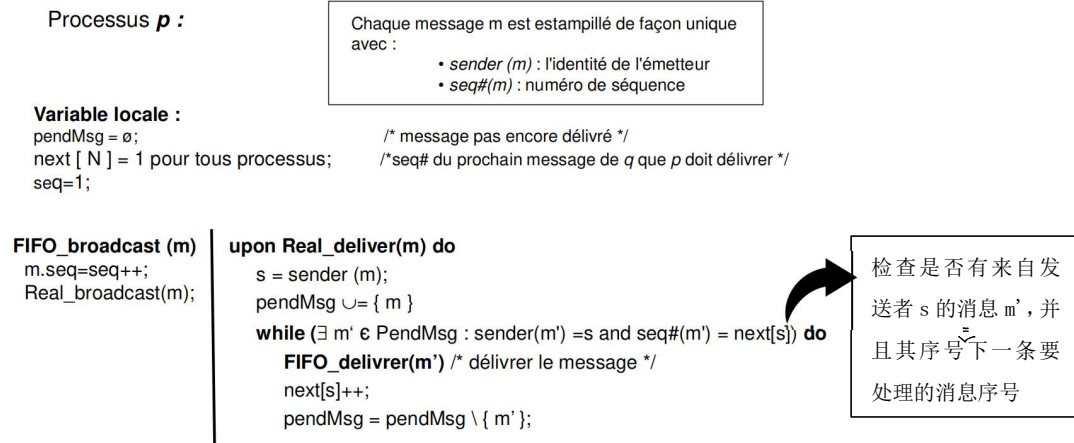
消息按发送顺序进行处理和交付

哪怕已经收到消息了，只要不符合 fifo (next 不符)，也不会交付，会等到接收到期望的 next 之后，再一起交付

先进先出顺序：如果一个成员先广播 m1，然后广播 m2，那么任何正确广播 m2 的成员都会在 m2 之前广播 m1。

有了可靠广播算法，就可以将其转化为先进先出广播算法。

算法：



Diffusion Causal = diffusion fiable + ordre Causal

按照信息传播的因果顺序传递信息

每一次广播不仅广播所需要广播的消息，还广播自上一次广播以来所有已经交付的消息

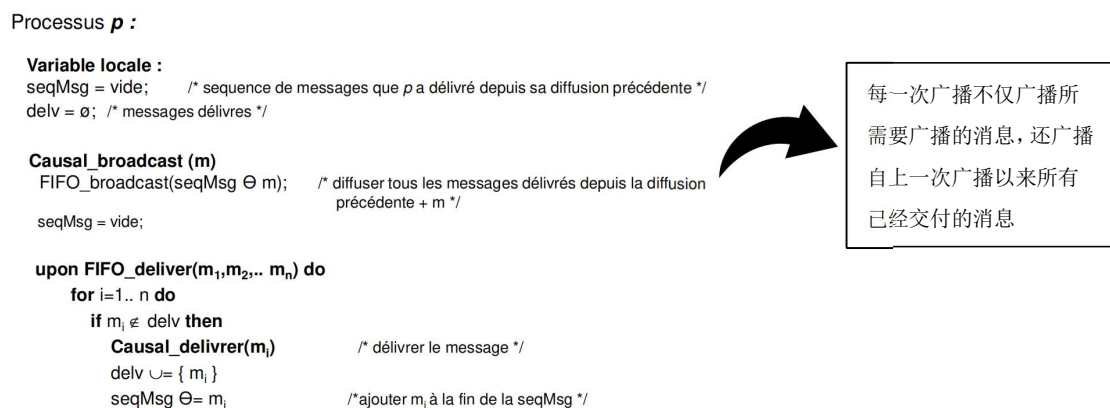
$broadcastp(m1) \rightarrow broadcastq(m2) \Leftrightarrow deliverp(m1) \rightarrow deliverq(m2)$

Causal Order $\rightarrow$ FIFO order

Fifo Order ! $\rightarrow$ Causal Order

FIFO 广播算法可以转换成因果广播算法：与每个广播消息一起传送因果在它之前的消息的历史。

算法：



• 优点：消息的传递不会在等待条件时延迟

• 缺点：消息较大

带向量时钟的因果广播算法

每个进程维持一个记录所有进程发送消息数量的消息队列，在发送消息之前，给自己在队列中的对应位置+1。收到消息的进程检查自己的消息队列与收到的消息队列是否只差消息发送者的1，是则读取消息并更新时钟，不是则等待差的消息。

区别：

普通因果广播：

通过广播前一次交付的所有消息来隐式地维护因果关系。

其他进程接收到时能通过这些消息来推导因果依赖关系。

带向量时钟的因果广播：

使用向量时钟（Vector Clocks）来显式追踪进程之间的因果依赖关系。

Processus P_i :

Variables locales :
 $HV[N] = \{0, 0, \dots, 0\}$
 $FA = \emptyset$

$HV[k]_m$ venant de P_j représente :

- $k = j$: le nombre de messages diffusés par P_j
- $k \neq j$: le nombre de diffusions de P_k délivrées par P_j avant diffusion de m .

Causal_Broadcast(m)

$HV[i] += 1$
estampiller m avec HV ;
envoyer m à tous les processus y compris p

Isis - Birman 91

Upon rcv($m, HV[k]_m$) :

$s = \text{sender}(m)$;
 $FA.\text{queue}(<m, HV[k]_m>)$
delay delivery of m **until**
 (1) $HV[s]_m = HV[s]_p + 1$ **and** (2) $HV[k]_m \leq HV[k]_p$ pour tout $k; k \neq s$
// D'autres réceptions se produisent pendant l'attente. On attend d'avoir délivré :
// 1- toutes les diffusions précédentes effectuées par s ,
// 2- toutes les diffusions délivrées par s avant la diffusion de m
 $FA.\text{dequeue}(<m, HV[k]_m>)$
 $\text{deliver}(m)$;
 $HV[s]_p += 1$;

- (1) : assure que p a délivré tous les messages provenant de s qui précèdent m
- (2) : assure que p a délivré tous les messages délivrés par s avant que celui-ci envoie m

原子广播

Diffusion atomique = diffusion fiable + ordre total

所有正确的进程都以相同的顺序传递相同的消息集。

全序：如果正确的进程 p 和 q 都传递消息 m 和 m' ，那么只有当 q 在 m' 之前传递 m 时， p 才会在 m' 之前传递 m 。

共识（Consensus）：分布式系统中多个节点（或进程）在某个值或决定上达成一致。

共识算法的核心目标：

一致性（Consistency）：所有正确工作的节点必须最终对某个值达成一致。

终止性（Termination）：所有节点必须在有限的时间内作出决定。

有效性（Validity）：节点决定的值必须来自某个节点提出的值，不能是虚构的。

容错性（Fault Tolerance）：算法能够在一定范围内的节点故障（如网络延迟或崩溃）下继续工作。常见的故障类型有拜占庭故障和崩溃故障。

在异步 + crash 故障模型下，实现原子广播和实现共识本质上一回事 —— 它们同样难，谁能解决其中一个，也就能解决另一个。

在一个异步系统（*système asynchrone*），如果存在“崩溃故障”（*pannes franches*），那么“原子广播等价于共识（*Consensus*）”。

如果能够实现原子广播算法，那么也能够实现共识。

实现共识，也能实现原子广播算法

“一个成，两个成，一个不成，两个不成”

FLP 定理：在一个完全异步的分布式系统中，如果存在崩溃故障（即节点可能会崩溃并且永远不再恢复），无法保证系统一定能够达成共识，无论使用什么算法。

在异步系统中，如果我们假设可能存在节点故障，ABCAST 是不可实现的。

ABCAST 是可实现(在有 n 个节点的系统中)条件:

带有类 P 或类 S 的故障检测器，并容忍 $n-1$ 个故障

带有类 $\diamond S$ 的故障检测器，并容忍 $n/2-1$ 个故障

使用有同步假设的时间控制的可靠广播协议

全序算法

ABCAST 协议

ABCAST 协议必须保证信息传递的顺序，并能容忍失败。

全序广播的实现：

一个或者多个序列器：

固定序列器（*Séquenceur Fixe*）：ABCAST 可以通过一个固定的序列器来确保消息的顺序。

这个序列器负责为每个消息分配一个序号，然后确保所有进程按相同的顺序接收消息。

移动序列器（*Séquenceur Mobile*）：除了固定序列器，另一种方案是移动序列器，即序列器的角色可以在不同节点之间切换。

发送者：

发送者的特权机制：有些实现可以通过给发送者一些特权，来控制消息顺序。

接受者：

接收者的协调：接收者可以通过一致同意的方式，来确保消息的顺序。

序列器：一个进程被选为序列器，它负责接收其他进程发送的消息，并为这些消息分配序号。

消息传递流程：

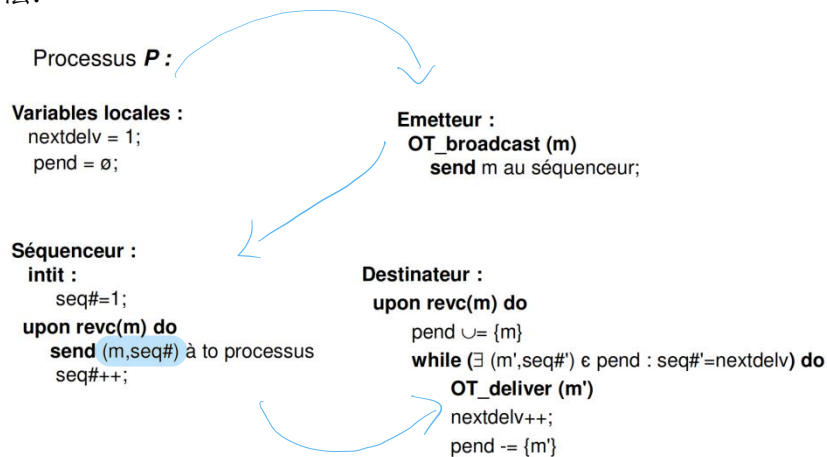
发送者先将消息发给序列器。

序列器给每条消息分配一个序号，然后将消息和序号一起广播给所有进程。

每个进程根据收到的消息序号，确保按序号递增的顺序交付消息。

固定序列器 Séquenceur fixe

算法:



接收者会检查
pend 集合中是
否存在序号为
nextdelv 的消
息

移动序列器

一组进程依次充当序列器

发送者会向所有序列器发送消息。

令牌在序列器之间循环，令牌携带：

序列号

已排序的消息列表

收到令牌后，序列器：

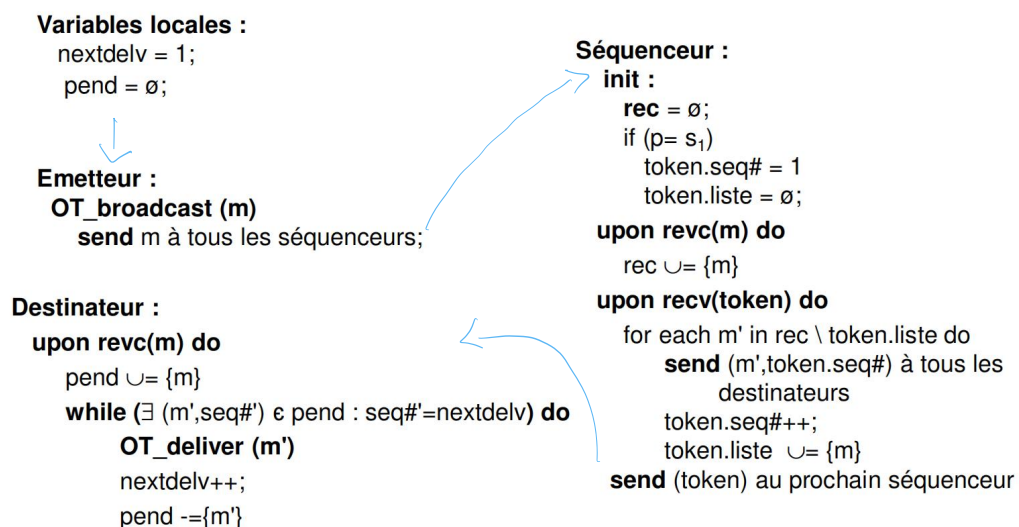
为所有尚未排序的消息分配序列号并将这些消息发送给收件人

将发送的消息添加到令牌列表中

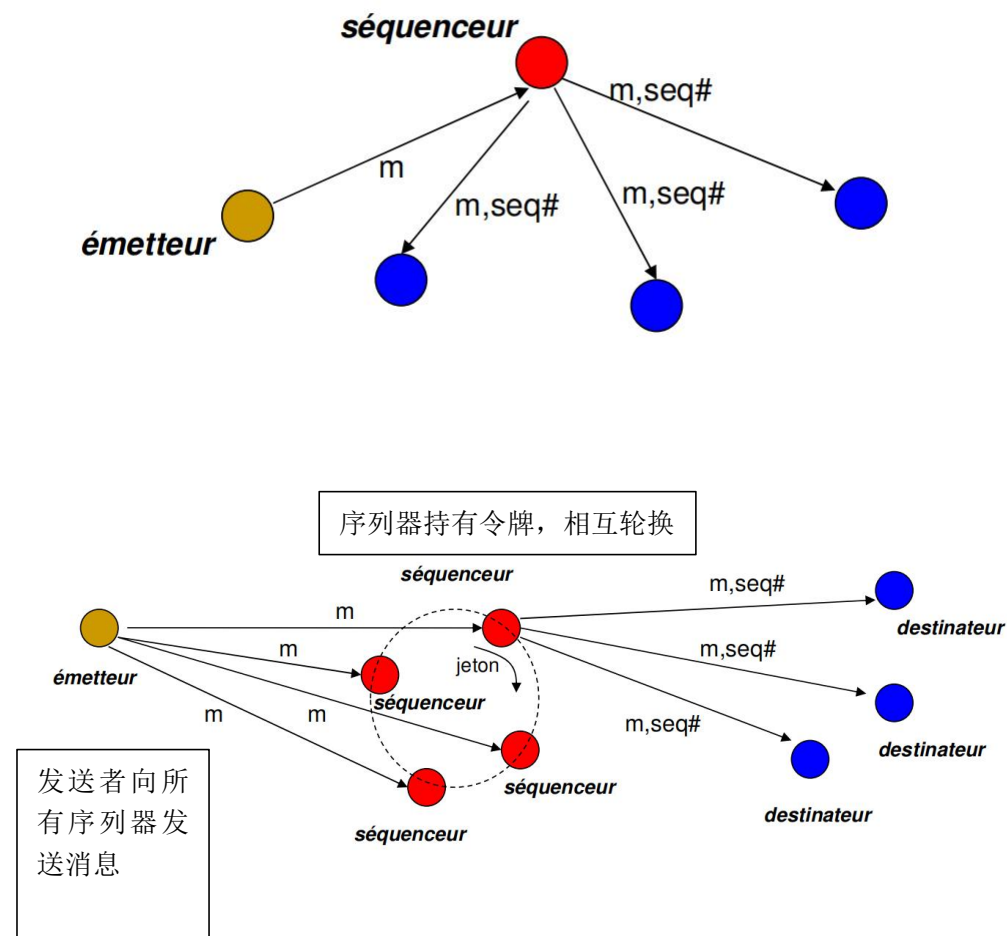
优点 负载分布[Urban 2000]

缺点 令牌大小 令牌流通成本

算法



区别



基于优先级的完全有序广播

令牌控制发送权利：在这种机制下，一个令牌（jeton）决定谁有权发送消息。只有持有令牌的发送者（émetteur）才能发送消息。

令牌在发送者之间循环：这个令牌在发送者之间传递，每个发送者在持有令牌时有权发送消息。令牌中包含下一个要发送消息的序列号（numéro de séquence）。

发送消息时必须持有令牌：

当一个发送者想要发送消息时，它必须等到它持有令牌。

持有令牌后，发送者可以给准备发送的消息分配一个序号。

发送完成后，发送者会将令牌传递给下一个发送者。

缺点（Inconvénients）：

需要知道所有的发送者

不公平性：由于令牌是在发送者之间循环传递的，一个发送者可能占有令牌较长时间，发送大量的消息，而其他发送者无法获得令牌进行发送。

算法：

Variables locales :

```
nextdelv = 1;  
pend = ∅;  
send_pend = ∅;
```

Destinateur :

```
upon revc(m) do  
  pend ∪= { m}  
  while (∃ (m',seq#') ∈ pend : seq#'=nextdelv) do  
    OT_deliver (m')  
    nextdelv++;  
    pend -= {m'}
```

Emetteur :

```
init :  
  send_pend = ∅;  
  if (p=s1)  
    token.seq# = 1  
    send (token) à s1 /*lui-même */
```

procedure OT_broadcast (m)

```
  send_pend ∪= {m}
```

upon recv(token) do

```
  for each m' in send_pend do  
    send (m',token.seq#) à tous les  
      destinateurs  
    token.seq#++;  
  send_pend = ∅;  
  send (token) au prochain émetteur
```

基于接收者达成共识的全序广播机制

工作原理（Principe）：

各个节点（接收者）协商为每条消息分配一个序列号。

为了达成共识，每次广播消息都需要经过两个阶段：

消息的广播和序列号建议的收集：消息首先被广播给所有接收者，接收者会提出对这个消息的序列号建议。

选择一个最终的序列号并将其广播给所有接收者：接收者根据收集到的所有建议，选出一个最终的序列号，并向其他接收者广播最终的序列号。

广播的进程给所有进程发送消息，所有收到消息的进程给出一个临时序列号，将消息发回广播进程，广播进程选择最大的序列号作为编号，然后再次广播，随后，收到最终序列号的进程可以正常交付消息。

算法：选最大的！

全序广播中的容错机制

故障检测器（DéTECTEURS de défaillance）：

故障检测器用于识别系统中可能发生的进程故障。

冗余（Redondance）：

冗余是通过复制或增加额外的组件来增加系统的可靠性。

消息稳定性（Stabilité des messages）：

消息的 k 稳定性（ k -stable）：

如果一个消息已经被 k 个进程接收到，那么这个消息被称为 k -stable。假设系统中最多有 f 个故障，那么如果一个消息被至少 $f+1$ 个进程接收到，即为 $(f+1)$ -stable，则可以保证该消息会被至少一个正确的进程处理，从而保证消息的最终交付。

消息丢失处理（Pertes de messages）：

为了处理消息丢失，全序广播通常会对消息进行编号（numérotation des messages），通过序号来跟踪每条消息是否被所有进程正确接收。丢失的消息可以通过序号来重新请求或恢复，从而确保不会因为消息丢失而打乱全序。

场景：某进程 **E** 想把消息 **m** 做**全序**扩散给组内所有接收者 **P₁...P_j**。每个接收者都维护一条**等待队列**（hold-back queue）。

1. 收到 **m** 时（在 **P_j** 侧）
 - 给 **m** 先打一个**临时序号**（numéro **provisoire**, tentative sequence number），把它标记为 **不可交付**（*non délivrable*），插入本地等待队列。
2. **P_j** 把自己的**临时序号**回给发送者 **E**，作为 **m** 的“候选最终序号”的**提议**（proposal）。
3. **E** 收齐**所有提议**后，选其中**最大的**那个作为 **m** 的**最终序号**（numéro **définitif**, final sequence number），并把“**m** 的 id，最终序号”再**广播**给所有接收者。
选“最大”的直觉：保证所有人对 **m** 的最终位置达成一致，且能和其他并发消息的最终序号形成一个**全序**。
4. 各接收者 **P_j** 收到**最终序号**后：
 - 在自己的等待队列中**按最终序号重排** **m**，并把 **m** 标为 **可交付**（*délivrable*）。
5. **P_j** **交付**：从队列**头部**开始，把所有已标为可交付且位于前端的消息依次 **deliver** 给应用层（保持全局一致顺序）。

这页右下角写着 **Birman-Joseph 87**：就是 Isis 系统里经典的“接收者一致”全序多播方案来源。

确定性广播 (deterministic broadcast) 不可扩展

1. 通信开销太大
2. 需要全局已知的成员列表
3. 强一致性的代价--全序，共识

2. Epidemic Broadcast 基本原理

- 当一个进程 **p** 要广播消息时，它随机选择 **k** 个**目标进程**（fanout），将消息发给他们。
- 收到消息的进程 **q** 会再次随机选择 **k** 个进程转发。
- 每个进程要维护一个 **消息缓存**，避免重复处理。
- 转发的步骤称为 **round**，通常设置一个 **最大轮数 r** 来避免无限传播。

👉 直观上就是“**像病毒传播一样**”，消息被节点一层层随机扩散。

3. 特点与性质

- **不保证完全可靠性**（不能确保所有节点一定收到），但在大规模系统中足够好。
- **可调参数**：
 - **fanout (k)**：每次转发的目标数，越大可靠性越高，但冗余也越大。
 - **rounds (r)**：最大转发轮数，越大传播范围越广，但通信开销也越大。
- **概率性性质**：
 - *Probabilistic validity*：以一定概率保证消息从发送者到所有正确节点。
 - *No duplication*：消息不会重复交付。
 - *No creation*：不会凭空生成消息。

2 级 4

5. Gossip 协议的几种策略

1. **Eager push**：节点一旦收到消息，立刻转发给随机的邻居。
2. **Pull**：节点定期向邻居询问“有没有我没见过的消息”，再请求拉取。
3. **Lazy push**：节点只先传播消息 ID，如果邻居没见过，就再请求完整消息。
4. **Hybrid**：先 push，再 pull 弥补遗漏。

6. 扩展：Partial view

- 在大规模系统里，每个节点不可能知道全局所有节点。
- 解决方案：只维护一个 **部分视图**（**partial view**），即邻居子集。
- 要保证：
 - **连通性**（**connectivity**）：整体图保持连通。
 - **度分布**（**degree distribution**）：每个节点的 in-degree / out-degree 不至于太小。
 - **平均路径长度**（**average path length**）：消息能快速扩散。
- 维护策略：
 - **Reactive**：遇到节点加入/崩溃时才更新。
 - **Cyclic**（**Cyclon 协议**）：周期性随机交换邻居表。
 - **Mixing**：在 gossip 消息中顺带携带已知的节点列表。

6. Gossip 在 Ad Hoc 网络中的应用

- 无线自组网（ad hoc network）：没有固定基础设施，节点广播只到一跳。
- **Gossip(p)**：
 - 源节点以概率 1 发送。
 - 其他节点第一次收到消息时，以概率 **p** 转发。
- **Gossip(p,k)** 改进：
 - 前 **k** 跳用概率 1 扩散，之后用概率 **p**。
 - 特殊情况：
 - $Gossip(1,1) \approx \text{flooding}$ （泛洪）
 - $Gossip(p,0)$ ：源节点也只以概率 **p** 扩散。

共享内存

定义：共享内存抽象是用于在多个进程之间共享存储的编程抽象，允许进程之间通过读写共享的存储空间来进行通信。

比消息传递更容易，不需要显式地发送消息。

在共享内存模型中，进程并发地访问数据对象或内存位置。

共享内存变量通常称为读写寄存器 read-write registers

互斥问题

互斥问题定义：在多进程系统中，进程共享资源时，必须确保在任何时刻，只有一个进程可以进入临界区（Critical Section, CS），以防止并发冲突。

安全性（Safety）：确保不会有两个进程同时进入临界区（Critical Section，简称 CS）。

无死锁（Deadlock-free）：如果一个进程想要进入它的临界区，总有进程能进入临界区，这保证了系统的全局进展。

注意：无死锁并不一定能防止饥饿（Starvation）。

防止饥饿（Starvation-freedom）：如果一个进程想要进入它的临界区，那么该进程最终一定能进入临界区。

面包店算法（Bakery Algorithm）

解决多进程的互斥问题，确保进程进入临界区的顺序遵循先来先服务（first-come, first-served）的原则，并且保证了饥饿自由。

读写寄存器

寄存器是共享变量的抽象概念

在多处理器系统中由硬件提供支持，这通常是通过共享的寄存器数组实现的。

在分布式系统中，没有共享设备的进程之间可以通过消息传递的方式来实现读写寄存器的功能。

可以通过分布式共享内存（Distributed Shared Memory, DSM）来模拟共享内存。

读写寄存器的用途：它们用于存储可以通过读写操作访问的值。

进程/线程之间通过调用这些读写操作来交换信息。

读写寄存器通常用于进程/线程之间的通信和同步，保证不同进程或线程可以可靠地读取和写入共享数据。

主要操作：

write(x, v)：写操作，表示将值 **v** 写入寄存器 **x**，并返回 **ok**，表示写入操作成功。

read(x)：读操作，表示从寄存器 **x** 读取当前的值，并返回这个值 **v**。

优先级/顺序关系：

如果两个操作 **o1** 和 **o2** 被执行：

o1 优先于 o2：当 **o1** 在 **o2** 调用之前结束时，称它们是顺序执行的（**sequential**）。

o1 与 o2 并发：当这两个操作没有明显的顺序时，称为并发。

安全性（Safety）：每次读操作都会返回最后一次写入的值。这意味着在没有并发冲突的情况下，读取的值应该总是最新的。

活性（Liveness）：每个操作最终都会完成。即使系统中有其他进程/线程在进行操作，保证每个操作最终能够执行并返回结果。

安全寄存器（Safe Registers）：保证一定的安全性要求，但可能不会处理所有并发情况。

常规寄存器（Regular Registers）：在并发情况下保证一定的顺序性。

原子寄存器（Atomic Registers）：保证最严格的并发访问顺序，读写操作看起来像是瞬时完成的。

- 一个节点修改了数据，其他节点可以及时发现。

内存一致性（Memory Coherence）：是系统正确执行内存操作的能力。具体来说，它是指系统在并发进程或线程执行的所有可能的内存操作交错（**interleaving**）中，确保内存一致性，意味着识别哪些序列是正确的。

内存一致性模型（Memory Consistency Model）：定义了允许的内存访问顺序集合，换句话说，它规定了在并发访问情况下，内存读写操作应该按照怎样的顺序执行，以保证正确性。

一致性准则：线性化是一种用于排序并发访问的强一致性模型。它要求：

所有操作看起来都是原子地、顺序地执行的，即每个操作都是瞬时完成的。

系统中必须模拟出全局时间尺度，所有进程（线程）需要就一个共同的全局顺序达成一致，这意味着每个进程（线程）看到的操作顺序是相同的。

其他一致性模型：相比线性化，一些一致性模型更加宽松，比如：

顺序一致性（**Sequential consistency**）*强一致性*

因果一致性（**Causal consistency**）

PRAM 一致性

弱一致性（**Weak consistency**）

线性化:

无论操作在实际中是并发还是非并发，只要它们的执行结果看起来像是按某个顺序（全序）依次进行的，我们就认为这个系统是线性化的。

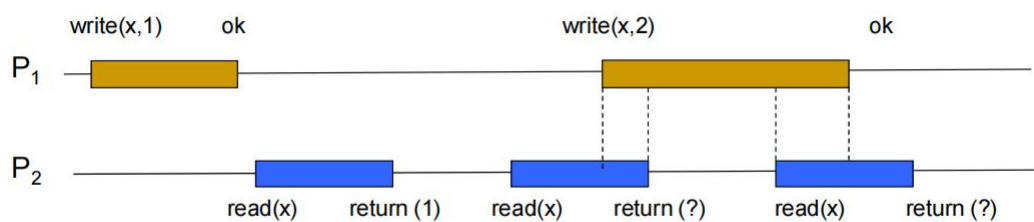
基于消息传递实现线性化）:

通过消息传递来实现线性化，其中每个操作（无论是读操作还是写操作）都被广播给其他进程。

安全寄存器 - Safe Register:

非重叠的读操作：如果读操作不与写操作重叠，则读操作返回最近写入的值。

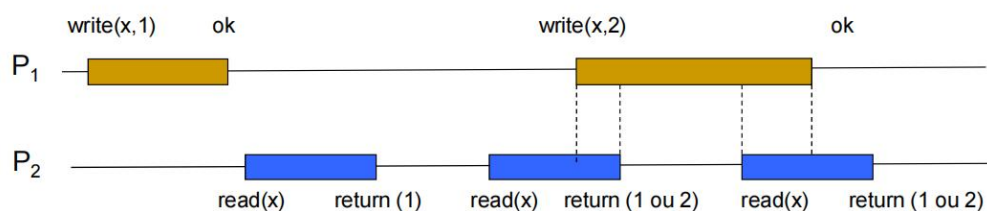
重叠的读操作：如果读操作与写操作重叠，则读操作可能返回任何值，不一定是最近写入的值。



常规寄存器 - Regular Register:

常规寄存器是安全寄存器的一种，并且具有额外的保证：

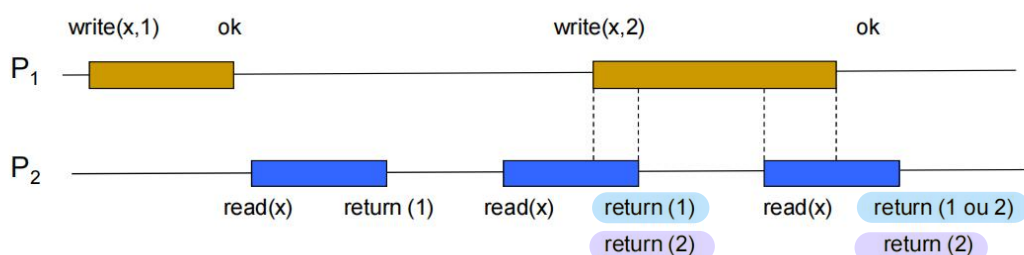
当读操作与写操作重叠时，读操作返回的是最近的写入值或并发写入的值。



原子寄存器 - Atomic Register:

原子寄存器是常规寄存器的一种，并且：

所有重叠的读写操作都是线性化的，这意味着系统中存在一个等价的全局顺序。



寄存器的特性：（一共 24 种）

语义（Semantics）：包括三类寄存器：安全寄存器（Safe）、常规寄存器（Regular）和原子寄存器（Atomic）。

值类型（Value）：寄存器可以存储二进制（Binary）值或整数（Integer）。

写入访问（Write Accesses）：分为单写者（Single-writer, SW）和多写者（Multi-writer, MW）。

读取访问（Read Accesses）：分为单读者（Single-reader, SR）和多读者（Multi-reader, MR）。

3
x
2
x
2
x
2
24

复杂寄存器可以通过组合多个简单寄存器来构建。每个简单寄存器负责处理写和读操作。

SRSW（Single-Reader Single-Writer）寄存器是一个只有一个写者和一个读者的寄存器，通过组合多个 SRSW 寄存器，可以构建一个 MRSW（Multi-Reader Single-Writer）寄存器，即有多个读者的单写者寄存器。

MRSW 安全寄存器

由 n 个 SRSW 安全寄存器，单个写者（P₀）， n 个读者（P₁...P_n）组成

多个读者不能同时访问同一个 SRSW 寄存器。

每个读者 P_i 只能访问其对应的寄存器 R_i。

写者 P₀ 写入相同的值到所有寄存器，这样每个读者最终可以读取相同的最新值。

MRSW 安全寄存器的行为：

安全性（Safe）：

当读操作和写操作在时间上不重叠时，读者能够读取到正确的值。

常规性（Regular）：

如果读操作和写操作发生了重叠，读者可能会返回正在写入的值或者之前的旧值。

写操作的行为：

P0 写者会将相同的值写入所有寄存器 R_i ，确保所有读者在不重叠的情况下，读取到相同的值。

MRSW 原子寄存器

由 n 个 SRSW 原子寄存器，单个写者（P0）， n 个读者（P1...Pn）组成

SRSW atomic registers of type $\langle \text{data}, \text{seq} \rangle$: $R_1 \dots R_n$

SRSW atomic registers of type $\langle \text{data}, \text{seq} \rangle$: $\text{LastRead_val}[n, n]$

Local Variables:

```
int seq, j, latest;  
<data, seq> LR[n+1]; /* last  
returned read value of other  
processes */
```

Write(val)

```
seq++;  
for j=1 to n  
     $R_j = \langle \text{val}, \text{seq} \rangle$ .
```

Read (val)

```
LR[0]= $R_i$   
for j = 1 to n  
    /* get latest value stored for  $P_i$  by  $P_j$  */  
     $\text{LR}[j] = \text{LastRead\_val}[j, i]$   
  
find max such that for all latest <> k  
     $\text{LR}[\text{latest}].\text{seq} \geq \text{LR}[k].\text{seq}$ ;  
  
for j = 1 to n  
     $\text{LastRead\_val}[i, j] = \text{LR}[\text{latest}]$ ;  
  
 $\text{val} = \text{LR}[\text{latest}]$   
return (val)
```

数据结构和变量

- **SRSW 原子寄存器** $R_1, \dots, R_n$ ：用于存储数据和对应的序列号，确保数据的顺序性。
- $\text{LastRead_val}[n][n]$ ：一个矩阵，用于记录每个读者上次读取的值，以便在多读者之间保持一致。
- **本地变量：**
 - seq ：序列号，用于标记每次写操作的顺序。
 - latest ：保存具有最大序列号的数据索引。
 - LR ：临时数组，用于保存每个读者的最新读取值。

写操作 Write(val)

1. **增加序列号**：每次写操作前，将 seq 递增，表示这是一个新的写入操作。
2. **写入每个寄存器**：遍历所有寄存器 R_j ，将 $\langle \text{val}, \text{seq} \rangle$ 写入 R_j ，确保每个寄存器都记录了最新的值和序列号。

读操作 Read(val)

1. **读取最新值**：
 - 读取当前进程的寄存器 R_i 的值并存入 $\text{LR}[0]$ 。
 - 遍历其他进程的 $\text{LastRead_val}[j, i]$ ，获取进程 P_j 已知的进程 P_i 的最新值，并存入 $\text{LR}[j]$ 。
2. **找到最大序列号**：
 - 在 LR 中寻找序列号最大的条目，并将其索引赋值给 latest 。这保证了最新的写操作的值被读取。
3. **更新所有的 LastRead_val**：
 - 遍历每个读者 j ，更新 $\text{LastRead_val}[i, j]$ 为 $\text{LR}[\text{latest}]$ ，即最新的值。这一步确保所有进程共享最新的读取信息。
4. **返回最新值**：
 - 将 $\text{LR}[\text{latest}]$ 的值赋给 val 并返回，确保读操作获得最新的值。

写操作 (Write(val))

1. **增加序列号**：每次写操作时，增加一个序列号 seq ，标记这是最新写入的版本。
2. **更新所有寄存器**：把新的值和序列号写到每个寄存器中，这样每个读者都能看到这个更新。

读操作 (Read(val))

1. **获取最新值**：读者去查找所有寄存器，看哪个寄存器里有**序列号最高的值**（也就是最新的值）。
2. **记录最新读取的值**：找到最新的值后，读者把这个值告诉其他读者，更新他们对最新值的记录。
3. **返回结果**：读者返回这个最新值，确保它读到的是当前最新的写入。

带故障恢复的 MRSW 寄存器

MRSW 寄存器仿真：

只有一个写者 P0 可以发起写操作，其他进程可以发起读操作。

使用完美故障检测器 (Perfect Failure Detector)：

每个进程都有一个局部变量来存储当前寄存器的值，这样即使有进程出现故障，每个进程依然有一个副本。

读一写全算法 (Read-One Write-All Algorithm)：

写者会更新所有没有被标记为故障的进程的值。

写操作完成的标志是所有进程都确认收到了新的值 (acknowledgement)。

读操作简单地返回存储在本地的值。

算法：每次写操作都会通过广播发送给所有进程，接收到确认后更新已写集合。当进程崩溃时，系统会移除崩溃进程，并检查是否所有正常进程已经确认了写操作。

Local Variables:

```
writeSet = ∅;  
reg = 0;  
correct = Π
```

Read ()

```
return reg;
```

Write(val)

```
Bebbroadcast <val>
```

Upon event Bebdelivery <val,j>

```
reg = val;  
send <j,ack>
```

Upon event crash <j>

```
correct = correct / {}  
if (correct C= writeSet)  
    writeSet = ∅;  
return <ok>;
```

Upon event reception

```
<ack,j>  
writeSet = writeSet U {}  
if (correct C= writeSet)  
    writeSet = ∅;  
return <ok>;
```

变量定义

- **writeSet**: 记录已经确认写入的节点集合。
- **reg**: 寄存器的值，用于保存写入的内容。
- **correct**: 表示当前系统中被认为是正常 (可用) 的节点集合 (假设系统中有多节点 Π)。

操作流程

1. **Read()**: 读取当前寄存器的值 **reg**。
 - 当调用 **Read()** 时，直接返回 **reg** 的值。
2. **Write(val)**: 写操作的过程
 - 写入某个值 **val** 时，首先调用 **Bebbroadcast(val)**，即广播这个写入值到所有节点。
 - 这一步的目的是让所有节点接收到新的写入值。
3. **Upon event Bebdelivery <val,j>**
 - 当某个节点收到广播的写入值 **<val,j>** 时 (即从节点 **j** 收到值 **val**)，它将 **reg** 值更新为 **val**。
 - 然后发送 **<ack,j>**，即发送一个确认消息 **ack** 给 **j**，表明自己已经收到了该写入值。

4. Upon event reception <ack,j>

- 当一个节点收到来自其他节点的确认 **<ack,j>** 消息时，将 **j** 加入到 **writeSet** 中。
- 检查 **writeSet** 和 **correct** 是否相等 (即所有正常节点都已确认写入)：
 - 如果 **writeSet** 包含了所有正常节点的确认 (即 **correct == writeSet**)，将 **writeSet** 清空，并返回 **<ok>**，表示写操作完成。

5. Upon event crash <j>

- 当检测到某个节点 **j** 崩溃时，将其从 **correct** 集合中移除。
- 然后再次检查 **writeSet** 和 **correct** 是否相等：
 - 如果相等，则清空 **writeSet**，返回 **<ok>** 表示写操作完成，因为系统认为写入已经被所有剩余正常节点确认。

常见的原子操作包括：

- **读（Read）和写（Write）**：这是最基本的原子操作，直接对寄存器进行读取和写入。
- **test-and-set（测试并设置）**：将寄存器 r 设置为 val ，并返回原来的值。
- **read-modify-write**：对寄存器 r 的当前值进行修改，应用函数 $f(r)$ ，然后将新值赋给 r ，并返回原来的值。
- **compare-and-swap（比较并交换）**：如果 r 的当前值等于 key ，则将其设置为 new ，否则保持不变。返回的是原来的值。
- **fetch-and-add**：将寄存器 r 的值增加 val ，并返回原来的值。
- **enqueue/dequeue**：对队列的原子操作，分别是将值加入队列和从队列中移除值。

Test-and-Set 算法：

确保在并发环境中实现互斥，只有一个进程可以访问临界区，其他进程必须等待。

该算法不能保证不会发生饥饿现象，某些进程可能长时间无法进入临界区。

- N processes
- Not starvation-free

Shared Variables:
register $reg = false$;

Local Variable:
 $blocked$;

```
function T&S (r:register, val:value): value;  
/* atomic*/  
temp = r;  
r=val;  
return (temp);
```

```
Entry_CS ()  
blocked=true;
```

```
repeat  
    blocked=T&S(reg,true);  
until blocked=false;
```

```
Exit_CS ():  
reg=false;
```

1. 共享变量和本地变量

- **Shared Variable** reg ：这是一个共享寄存器，用来标识临界区是否被占用。初始化为 $false$ ，表示没有进程在临界区。
- **Local Variable** $blocked$ ：这是一个本地变量，用于当前进程的状态，指示是否被阻塞。

2. Test-and-Set 函数 $T\&S(r: register, val: value): value$

- **T&S** 是一种原子操作，用来实现锁。
- 功能：将寄存器 r 的当前值暂存到 $temp$ ，然后把 r 设置为 val ，最后返回 $temp$ 。
 - 如果 r 原本是 $false$ ， $T\&S$ 返回 $false$ 并将 r 设置为 $true$ 。
 - 如果 r 已经是 $true$ ， $T\&S$ 直接返回 $true$ ，表示锁已经被其他进程占用。
- 由于 $T\&S$ 操作是原子的，所以可以用来保证只有一个进程成功修改 reg 。

3. 进入临界区 $Entry_CS()$

- 先将 $blocked$ 设为 $true$ ，表示当前进程准备进入临界区。
- 进入一个循环：调用 $T\&S(reg, true)$ ，将 reg 设置为 $true$ 。
 - 如果 $T\&S$ 返回 $false$ ，说明 reg 原来是 $false$ ，当前进程成功占用锁，可以退出循环。
 - 如果 $T\&S$ 返回 $true$ ，说明 reg 已被其他进程占用，当前进程继续循环等待。

4. 退出临界区 $Exit_CS()$

- 当进程执行完临界区的操作后，将 reg 重新设置为 $false$ ，释放锁，使其他进程可以进入临界区。

共享原子对象 是一种数据结构，它可以被系统中的进程或线程并发调用。它保证这些操作是原子执行的。

- **寄存器（Registers）**：支持读写操作的共享寄存器。
- **Test-and-Set 对象**：支持写操作和测试并设置（test-and-set）的共享寄存器。
- **Read-modify-write 对象**：支持读-修改-写操作的共享寄存器。
- **Compare-and-Swap 对象**：支持比较并交换（compare-and-swap）操作的共享寄存器。
- **队列（Queue）**：支持入队（enqueue）和出队（dequeue）的共享队列。

寄存器：故障问题

如果某个进程在执行 write 操作时崩溃，则未完成的 write 操作会与随后发生的任何 read 操作并发，可能导致一致性问题。

任意进程发起的 read 或 write 操作，只要没有崩溃，最终都会返回。

- **阻塞操作 (Blocking operations)**：如果某个进程（或线程）延迟或崩溃，可能会阻止其他进程继续前进。例如，互斥锁（mutex）和生产者-消费者问题。
- **非阻塞操作 (Non-blocking operations)**：即使某个进程（或线程）延迟或崩溃，也不会阻止其他进程继续前进。在这种情况下，竞争共享资源的进程不会因为其他进程的执行而无限期推迟。

非阻塞操作 (Non-blocking Operations)

- 非阻塞操作保证即使某个进程出现故障，其他进程仍然能够继续前进。
- **等待自由 (Wait-freedom)**：这是最强的非阻塞保证，意味着所有进程都能在有限步内完成操作，不会因为其他进程的延迟或崩溃而被阻塞。
- **共识问题 (Consensus Problem)**：在异步系统中达成一致性共识是一个经典的难题，特别是在面对进程崩溃时，FLP 不可能性证明说明了在某些条件下共识是不可能实现的。

共识问题 (Consensus)：

共识问题是分布式系统中的一个经典问题，主要是指多个进程（或节点）需要就某个值达成一致。在并发编程或分布式系统中，所有节点必须选择同一个值。

为什么共识在崩溃失效模型下是不可解的？：

在崩溃失效模型中，如果一个进程崩溃了，其他进程可能不知道该进程是崩溃了，还是只是延迟了操作。在这种情况下，系统无法保证所有进程都能就一个值达成共识。

可以达成共识的对象：

- 尽管在异步模型和崩溃失效条件下，共识问题一般不可解，但某些特殊对象(objects)可以在 wait-free 算法下达成共识。
 - **Wait-free 算法**：这种算法确保每个进程在有限步骤内完成其操作，而不会因为其他进程的延迟或崩溃而被无限期阻塞。
 - 例如，**test-and-set** 对象、**compare-and-swap** 对象以及 **FIFO 队列** 对象，提供了比普通的读/写原子寄存器更强的同步能力，因此可以在 n 个进程中使用 wait-free 算法达成共识。

提供更强同步能力的对象：

- 这些对象比普通的安全（safe）、常规（regular）或原子（atomic）对象有更强的同步能力，因此可以允许进程在它们上面执行 wait-free 的操作，从而帮助解决共识问题。
- test-and-set 和 compare-and-swap 操作可以确保多个进程间的互斥访问，同时避免某些情况下的死锁或无限等待。

Consensus Number (一致性数)

- 一致性数 kkk 是衡量某个对象类型能够支持多少个进程之间的 wait-free 一致性解决方案的一个指标。

Consensus number of some objects (某些对象的一致性数)

- 一致性数为 1 的对象只能解决 2 个进程之间的共识问题，这类对象包括：Register（寄存器）。
- 一致性数为 2 的对象可以解决 3 个进程的共识问题，这类对象包括：FIFO Queue（FIFO 队列），Test-and-Set（测试并设置），Fetch-and-Add（取并加）。
- 一致性数为无穷大 (∞) 的对象可以解决任意数量的进程的共识问题，例如 Compare & Swap（比较并交换）。

Test-and-Set 对象实现无等待共识

• Two processes

x_i : initial choice

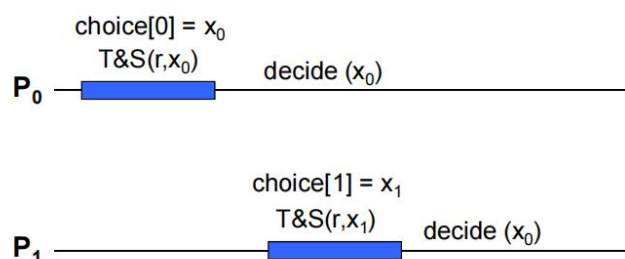
Shared Variables :

test-and-set reg $r = \perp$;
r/w reg $\text{choice}[2] = \{\perp, \perp\}$;

Local Variables:

int val;

```
choice[i] =  $x_i$ ;  
val = T&S( $r, x_i$ )  
if (val =  $\perp$ )  
    decide ( $x_i$ )  
else  
    decide (choice[(i+1)%2]);
```



It does not work with more than 2 processes:
which value to chose from *choice[]* ? non

算法步骤

- 每个进程设置自己的选择：
 - P_0 将 $\text{choice}[0]$ 设为 x_0 （即它自己的初始选择）。
 - P_1 将 $\text{choice}[1]$ 设为 x_1 （它自己的初始选择）。
- Test-and-Set 操作：
 - 每个进程 P_i 调用 $\text{T\&S}(r, x_i)$ 来尝试将 r 设置为它自己的选择 x_i 。
 - 如果 $\text{T\&S}$ 返回 $\perp$ （表示 r 之前是空的），那么当前进程成功设置了 r 的值，可以直接决策 x_i 作为最终选择。
 - 如果 $\text{T\&S}$ 返回了一个值（非 $\perp$ ），说明另一个进程已经设置了 r 的值，那么当前进程选择 $\text{choice}[(i+1) \% 2]$ ，即另一个进程的选择，作为最终决策。
- 最终决策：
 - 每个进程在 decide 阶段，选择的结果要么是它自己的初始选择，要么是另一个进程的选择，确保两者达成一致的决策。
 - 比如，若 P_0 成功设置了 r ，则 P_0 和 P_1 最终都会选择 x_0 作为共同的决策。

变量说明

- 输入 x_i ：每个进程 P_0 和 P_1 的初始选择，即它们各自希望选择的值。
- 共享变量：
 - Test-and-Set 寄存器 r ：用于进行原子操作，初始值为 $\perp$ （表示“空”或“未设置”状态）。
 - 读/写寄存器 $\text{choice}[2]$ ：一个数组， $\text{choice}[0]$ 和 $\text{choice}[1]$ 用于存储两个进程的初始选择，初始值为 $\perp$ 。
- 本地变量 val ：每个进程使用它来存储 Test-and-Set 操作的结果。

- 输入 x_i : 每个进程 P_0 和 P_1 的初始选择值, 即它们各自希望选择的值。

- 共享变量:

- 队列寄存器 Q : 用于管理进程的访问顺序, 初始状态中含有一个值 $\langle \perp \rangle$ 。这表示最初 p_0 可以访问队列。
- 读/写寄存器 $choice[2]$: 一个数组, 用于存储每个进程的初始选择, 初始值为 $\perp$ 。

- 本地变量 $temp$: 用于存储 $dequeue(Q)$ 操作的结果。

FIFO 队列对象实现无等待共识

Two processes

x_i : initial choice

Shared Variables :

```
queue reg Q= <0> /* Q initializ   */
r/w reg choice[2] = {  ,   };
```

Local Variables:

```
int temp;
```

$choice[i] = x_i$;

```
temp= dequeue(Q) ;
```

```
if ( temp ==   )
```

```
/* queue empty*/
```

```
decide (choice[(i+1)%2]);
```

```
else
```

```
decide ( $x_i$ )
```

Compare-and-Set 对象实现无等待共识

∞ processes

x_i : initial choice

Shared Variables :

compare-and-swap reg;

Local Variables:

```
int temp;
```

```
temp= C&S(reg,   ,  $x_i$ );
```

```
if (temp =   )
```

```
decide ( $x_i$ )
```

```
else
```

```
decide (temp);
```

function C&S (r:register; key, new: value): value;

```
/* atomic*/
```

```
temp = r;
```

```
if (key=temp) r=new;
```

```
return (temp);
```

P_0 C&S($r, \perp, x_0$)

decide (x_0)

P_1 C&S($r, \perp, x_1$)

decide (x_0)

P_2 C&S($r, \perp, x_2$)

decide (x_0)

P_3

...

变量说明

- 输入 x_i : 每个进程的初始选择值, 即它们各自希望选择的值。

- 共享变量:

- Compare-and-Swap 寄存器 reg : 用于管理多个进程的访问和决策一致性。初始值为 $\perp$ (表示“空”或“未设置”状态)。
- 本地变量 $temp$: 用于存储 C&S 操作的结果。

Compare-and-Swap (C&S) 操作解释

- C&S 函数: $C\&S(r, key, new)$ 是一个原子操作, 用于比较并更新寄存器 r 。

- 首先, 将 r 的当前值存储在 $temp$ 中。

- 然后, 检查 r 的值是否等于 key (在这里初始为 $\perp$)。

- 如果 $r == key$, 那么将 r 的值更新为 new , 即当前进程的选择。

- 最后, 返回 $temp$, 即 r 的旧值。

算法步骤

1. 每个进程执行 C&S 操作:

- 每个进程 P_i 调用 $temp = C\&S(reg, \perp, x_i)$, 尝试将 reg 设置为自己的选择 x_i , 条件是 reg 之前的值为 $\perp$ 。

- 如果 reg 的值确实为 $\perp$, 则 C&S 成功, reg 被设置为 x_i , 并且 $temp$ 的值为 $\perp$ 。

- 如果 reg 的值已经被其他进程设置过, C&S 不会修改 reg , 返回已有的值。

2. 决策逻辑:

- 如果 $temp == \perp$, 说明当前进程成功将自己的选择设置为 reg 的值, 因此它选择 x_i 作为决策。

- 如果 $temp \neq \perp$, 说明另一个进程已经设置了 reg 的值 (比如, x_0), 因此当前进程选择 $temp$ 中的值作为决策, 以保持一致性。

算法步骤

1. 每个进程设置自己的选择:

- P_0 将 $choice[0]$ 设为 x_0 , 表示其初始选择。

- P_1 将 $choice[1]$ 设为 x_1 , 表示其初始选择。

2. 出队操作:

- 每个进程执行 $temp = dequeue(Q)$, 从队列 Q 中取出一个值, 并将其赋给 $temp$ 。

- 如果 $temp$ 为 $\perp$, 表示队列为空, 即当前进程不是第一个访问的进程。

- 如果 $temp$ 不是 $\perp$, 表示该进程是第一个访问的进程, 可以使用自己的选择。

3. 决策逻辑:

- 队列为空 ($temp == \perp$): 如果当前进程不是第一个访问队列的进程, 它会选择 $choice[(i+1)\%2]$, 即另一个进程的选择。

- 队列不为空: 如果当前进程是第一个访问队列的进程, 它会选择自己的选择 $choice[i]$ 作为最终决策。

- test-and-set: 靠“谁第一个拿到锁”决定答案。

- FIFO 队列: 靠“谁第一个入队”决定答案。

- CAS: 靠“谁第一个成功写共享变量”决定答案。

三者本质一样: 第一个成功的人决定最终共识, 其他人只能跟随。

内存一致性模型

Linearizability (线性化) = 严格一致性 / 原子一致性

主要类型:

- Sequential Consistency (顺序一致性)
- Causal Consistency (因果一致性)
- PRAM Consistency (管道寄存器内存一致性)

基于同步变量的模型:

- Weak Consistency (弱一致性)
- Entry Consistency (入口一致性)
- Release Consistency (释放一致性)
- Lazy Release Consistency (惰性释放一致性):

顺序一致性

- 在顺序一致性模型下, 执行结果应该等同于所有处理器的操作按某种顺序执行。
- 每个处理器的操作必须按照它的程序顺序执行。
- 即使操作来自不同的处理器, 它们的执行顺序可以交错, 但所有处理器看到的顺序必须一致。
- 即使两个操作在全局时间上没有重叠, 它们的顺序在所有进程中看起来必须是一致的, 甚至可能是反向的。

因果一致性

因果相关的写操作必须在所有进程中按照相同的顺序观察到, 而并发写操作则可以按不同的顺序观察到。

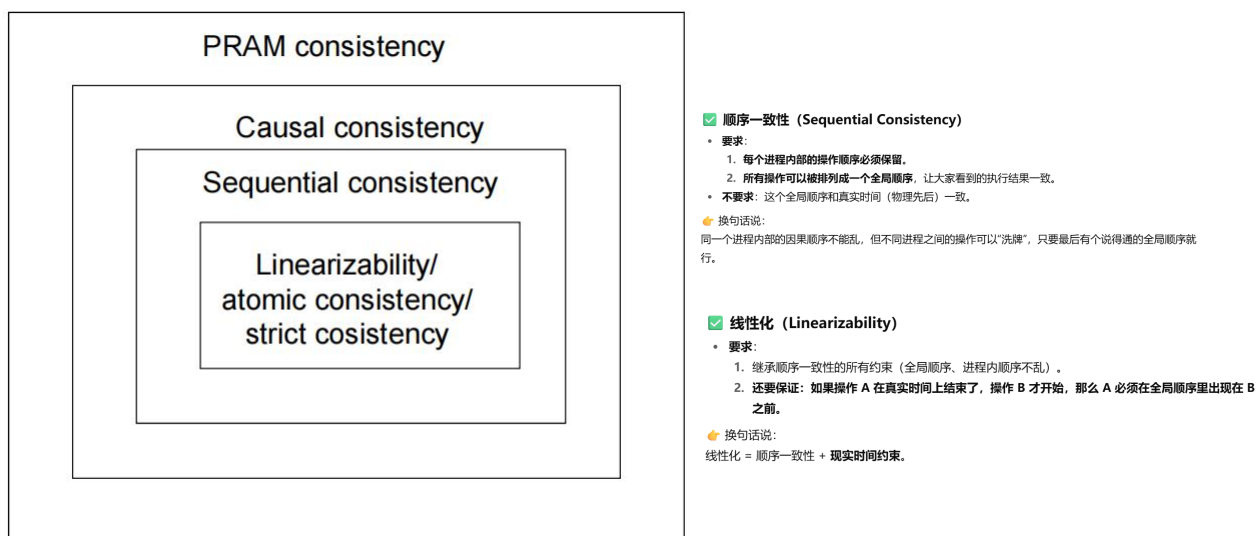
- 进程的本地事件顺序定义了局部的因果顺序。
- 写操作先于读操作, 如果读操作读取了写操作写入的值, 那么这个写操作与这个读操作因果相关。
- 通过这些规则的传递性, 可以确定全局的因果顺序。

PPAM 一致性

主要强调单个处理器的写操作必须以其发出的顺序被其他处理器感知, 但不同处理器的写操作可以以不同顺序被感知。

PRAM 一致性保证每个处理器的本地写操作顺序不会被打乱, 但跨处理器的写操作顺序可能是不同的。

包含关系（强弱关系）：



1. Linearizability（线性化）/Atomic consistency（原子一致性）/Strict consistency（严格一致性）：

- 这是最严格的模型，要求所有操作看起来像是瞬时发生的，并且与现实时间相符。它确保所有进程在任何时刻都能观察到一个全局一致的顺序。

2. Sequential consistency（顺序一致性）：

- 它允许多个进程的操作顺序可以被交错执行，但要求所有进程看到的操作顺序必须是一样的。与线性化相比，顺序一致性不需要考虑实际的物理时间。

3. Causal consistency（因果一致性）：

- 只要求因果相关的写操作必须被所有进程以相同顺序看到，而对于并发的写操作，它们可以以不同顺序被不同的进程看到。

4. PRAM consistency（管道RAM一致性）：

- 要求单个处理器的写操作必须按照发出的顺序被其他处理器感知，但不同处理器的写操作顺序可以不同。

Fast Userspace Mutex

一种高效的`用户空间互斥锁`，用于共享内存同步。它的设计是尽量减少进入内核的操作。

当多个线程对共享变量没有竞争时，操作是在用户空间中完成的；如果发生竞争，则需要使用内核提供的 `futex` 系统调用来管理同步。

无竞争（No contention）时：操作仅在用户空间完成，不需要涉及内核服务。

有竞争时：涉及的进程或线程需要 `futex` 系统调用来进行等待或唤醒。

算法：

```
#define futex_wait(addr, val) syscall(SYS_futex, addr, FUTEX_WAIT, val, NULL)
#define futex_wakeup(addr, nb) syscall(SYS_futex, addr, FUTEX_WAKE, nb)
```

```
void LOCK(int *f) {
    int old;
    while (1) {
        old = compare_and_swap(f, 0, 1); // opération atomique
        if (old == 0) // The futex variable was free. Now, it is set to 1
            return;
        else
            futex_wait(f, 1); // LOCK
    }
}
```

```
void UNLOCK(int *f) {
    fetch_and_sub(f, 1); // opération atomique
    futex_wakeup(f, 1); // UNLOCK
}
```

shared int futex_var;

```
void *thread_exmple (void *p) {

    while(1) {
        ...
        LOCK(&futex_var);
        ....
        UNLOCK(&futex_var);
    }
}
```

1. Futex 系统调用定义

代码的前两行定义了两个宏：

- `futex_wait(addr, val)`：通过 `syscall` 调用 `SYS_futex`，让线程在 `addr` 上等待值为 `val`。如果条件不满足，线程会阻塞。
- `futex_wakeup(addr, nb)`：通过 `syscall` 调用 `SYS_futex`，唤醒在 `addr` 上等待的线程，唤醒数量为 `nb`。

这两个宏分别用于实现 Futex 的阻塞和唤醒操作。

2. LOCK 函数

`LOCK` 函数用于实现加锁操作。其逻辑如下：

- 定义一个局部变量 `old`。
- 进入一个无限循环 `while (1)`，直到成功获取锁才返回。
 1. **原子性操作**：调用 `compare_and_swap(f, 0, 1)` 尝试将 `*f` 从 `0` 设置为 `1`，表示加锁。
 - 如果 `*f` 为 `0`（表示锁是空闲的），`compare_and_swap` 会将其设置为 `1` 并返回 `0`。
 - 如果 `*f` 不为 `0`，表示锁已被占用，`compare_and_swap` 返回当前值（即非 `0`）。
 2. **判断是否成功获取锁**：如果 `old == 0`，表示锁之前是空闲的，现在成功将其设置为 `1`，可以直接 `return` 返回，表示加锁成功。
 3. **等待**：如果 `old != 0`，表示锁已被其他线程占用，当前线程调用 `futex_wait(f, 1)` 进入阻塞状态，直到被唤醒。

3. UNLOCK 函数

`UNLOCK` 函数用于解锁操作。其逻辑如下：

- 使用原子操作 `fetch_and_sub(f, 1)`，将 `*f` 的值减 `1`，表示解锁。
 - 当 `*f` 从 `1` 变为 `0` 时，锁释放，其他等待的线程可以尝试获取锁。
- 调用 `futex_wakeup(f, 1)` 唤醒一个在 `f` 上等待的线程。

4. thread_example 函数

这是一个线程函数的例子，展示了如何使用 `LOCK` 和 `UNLOCK` 函数来控制对共享资源的访问。

- 定义一个共享变量 `futex_var`。
- 在线程循环中调用 `LOCK(&futex_var)` 来获取锁，在临界区（可能是一些操作代码）后调用 `UNLOCK(&futex_var)` 释放锁。

总结

- **加锁**：`LOCK` 函数使用 `compare_and_swap` 尝试获取锁，若获取失败则调用 `futex_wait` 阻塞，等待其他线程释放锁。
- **解锁**：`UNLOCK` 函数使用 `fetch_and_sub` 解锁，并调用 `futex_wakeup` 唤醒在该锁上等待的线程。
- **高效同步**：这种基于 Futex 的锁机制在用户态和内核态之间快速切换，实现了高效的线程同步，在锁空闲时避免进入内核态，从而减少开销。

共识和故障检测

Le consensus = dénominateur commun

共识 = 共同标准

- 有效性：如果进程决定了 v ，那么 v 就是建议值
- 终止：所有正确的进程最终做出决定
- 统一协议：两个正确的进程不能做出不同的决定

两个原语

- 提议 (v)：调用进程提议一个初始值 v
- 决定 (v)：调用进程决定 v

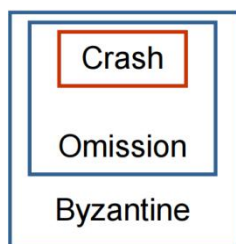
两种进程：

成功：在整个执行过程中不失败

失败：没成功

进程：

- 坦率（崩溃）：故障进程不再永久发送或接收消息=故障静默-故障静默、故障停止变体（可见故障）
- 遗漏：短暂
- 暂时：太早或太晚
- 拜占庭：恶意



信道：

- 可靠：如果 p 向 q 发送 (m)，并且 q 是正确的，那么 q 将收到 m
- 准可靠：如果 p 向 q 发送 (m)，且 p 和 q 都正确，则 q 将收到 m
- 公平损失：如果一个正确的进程向 q 发送信息 m 无穷多次，那么 q 将收到 m

互连假设 Interconnexion

1. $\Pi = \{p1, p2, \dots, pN\}$
 - 这里的 Π 表示系统中的进程集合。
 - 一共有 N 个进程，分别记作 $p1, p2, \dots, pN$ 。
2. **communicent par passage de messages**
 - 这些进程之间 **通过消息传递来通信** (Message Passing)。
 - ⚠️ 说明这是 **分布式系统的模型**：没有共享内存，只有发消息来交流。
3. **Graphe complet (完全图)**
 - 通信拓扑是一个 **完全图**：
 - 也就是说，**任意两个进程之间都可以直接发送消息**。
 - 没有限制“谁能跟谁说话”。

时态模型

- 关于信息传输和处理速度的假设

同步模型:

- 关于传输时间的边界 Δ : 如果进程 p 在 t 时向 q 发送信息, 则 q 在 $t+\Delta$ 之前收到信息。
- 进程相对速度的边界 ϕ : 如果最快的进程处理一条信息需要 x 个单位的时间, 那么最慢的进程处理同一条信息所需的时间不能超过 $x\phi$ 。

传输时间无限制

- 对相对进程速度没有限制

=> 无法区分缓慢进程和“崩溃”进程

部分同步系统

- 介于同步模型和异步模型之间的中间模型 (32 个模型)
- 同步模型的边界 Δ 和 ϕ :

1. 存在但未知, 或
2. 已知, 但在称为 GST (全局稳定时间) 的时间 T 之后出现。
 - 在 GST 之前, 系统不稳定 (无界限)
 - GST 之后, 系统稳定 (有界限)
 - GST 未知

(GST 之后, 系统表现得像一个同步系统, 并且系统的行为可预测)

维度	取值	说明
消息延迟 Δ	4 }	已知上界
		我们不知道最大延迟
		存在但未知
		有最大延迟, 但我们不知道是多少
	1 }	GST 后已知
		从 GST 开始有个已知上界
	2 }	GST 后未知
		从 GST 开始有上界, 但不知道是多少
处理速度 ϕ	4 }	同 Δ
时钟漂移 θ	2 }	有界
		时钟同步性可保证
		无界
		时钟可能完全乱漂

$$4 \times 4 \times 2 = 32$$

FLP 不可能性定理

在异步, 可靠网络, 只有一进程发生故障的情况下无法通过确定性的方法解决共识问题

绕过 FLP 不可能性定理

- **改变问题:** 通过放松一致性要求 (如 k -agreement) 来绕过 FLP 定理的限制。
- **部分同步系统:** 假设系统在某个时候进入同步状态, 从而使共识成为可能。
- **不完美共识:** 采用概率性的算法 (如 Paxos) 来在异步系统中达成共识。
- **不可靠的故障检测器:** 通过引入不完美的故障检测器, 帮助系统在异步环境中实现一致性。

CHT 96

不可靠故障检测器 [CHT 96]

- 1991 年引入
- 每个节点上的本地 Oracle (类似于一个探测器或监视器), 它会尝试判断其他节点的状态。
- 提供疑似故障进程列表
- 不可靠信息
 - 可能出现错误怀疑

完整性: 应将失败的流程检测为失败

准确性: 不应将正确的流程视为失败

探测器质量

完整性

- 强：所有正确流程都会瞬间怀疑任何错误流程。
- 弱：任何错误流程都会瞬间被正确流程所怀疑。

准确性：

- 强：没有正确流程被怀疑
- 弱：至少有一个正确的流程从未被怀疑过
- 最后强：在某一时刻，任何正确的程序都不再被任何正确的程序怀疑
- 最后弱：至少有一个正确过程没有被任何正确过程怀疑。

	Justesse			
	Forte	Faible	Finalement forte	Finalement faible
Complétude forte	P	S	$\Diamond P$	$\Diamond S$
Complétude faible	Q	W	$\Diamond Q$	$\Diamond W$

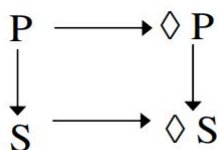
故障检测器分类

检测器分为四类，依据其完整性和准确性：

- **P**：强完整性 + 强准确性。
- **S**：强完整性 + 弱准确性。
- **Q**：弱完整性 + 强准确性。
- **W**：弱完整性 + 弱准确性。

带菱形的表示最终 xxx

强完备性和弱完备性是等价的（我们可以从弱完备性构建强完备性）



P 强于 S 和 $\Diamond P$

$\Diamond P$ 强于 $\Diamond S$

S 强于 $\Diamond S$

强的检测器可以推导出弱的检测器

检测器的强度反映了它对系统故障的反应和识别能力，较强的检测器在更广泛的条件下能够更可靠、更准确地检测到故障。

- **强检测器能推出弱检测器**：因为强的包含弱的（精确天气预报包含笨天气预报的信息）。
- **弱的能构建强的**：虽然弱检测器信息少，但如果配合算法（比如选一个领袖、多数投票），也能“升级”为足够强的能力去解决共识。

强完备性和弱完备性是等价的。
在分布式故障检测器的场景中可以实现彼此的转换。

如果一个系统具备强完备性，那么它必然具备弱完备性；而具备弱完备性时，也可以在一定条件下转化为具备强完备性。

探测器和共识

- 可通过 Ω 达成共识

- Ω 是最弱的检测器，能以大多数正确进程解决共识（最小化）问题

◇ w 无法解决共识问题

- FLP => 不可能异步实现

终极领导者

- Ω 故障检测器

Ω ：故障检测器，其输出是假定正确的单一过程

q 是 W 在 t 时刻的输出： P 信任 t 时刻的 q

Ω 确保：

有一天，所有正确的进程都会信任同一个正确的进程。

Ω 和 $<\Omega$ 等价

Σ = Quorum Failure Detector，它提供的是一组集合（quorums），这些集合有两个性质：

1. 交集性 (Intersection property)

任意两个正确进程得到的集合必须有交集（即使他们拿到的集合不一样，但至少有一个进程在两边都出现）。

✪ 保证了不会形成完全独立的“小团体”。

2. 完整性 (Completeness)

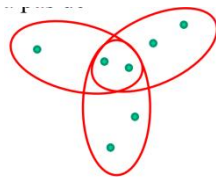
最终，所有集合只包含正确进程（挂掉的会被排除）。

具有 $n-1$ 个错误的最小检测器

• 错误检测器 Σ —提出正确进程的列表

• 交集： Σ 提出的列表中始终存在非空交集

• 最终完整性：最终，列表中不存在错误进程



• 表格纵轴 (Models)

- Shared memory (共享内存模型)：进程通过共享变量通信（类似多线程读写内存）。
- Message passing (消息传递模型)：进程之间靠发消息通信（类似网络分布式系统）。

这两个模型对“最弱需要的故障检测器”要求不同。

• 表格横轴 (Problems)

- Consensus：共识问题，所有进程最终必须决定同一个值。
- k-set agreement：允许最终最多有 k 个不同的值。
- Set agreement：广义集合一致性问题。
- Eventual consistency：最终一致性（例如存储系统里的 eventual consistency）。

• (Ω, Σ) 实现具有 $n-1$ 个错误的共识的最弱检测器

最小错误检查器

Problems Models	Consensus	k-set agreement	set agreement	Eventual consistency
Shared memory	Ω [LH94]	k-anti- Ω [GK09]	anti- Ω [Z10]	
Message passing	(Ω, Σ) [DFG10]	?	$\mathcal{L}$ [DFGT08]	Ω [DKGPS15]

• 表格里的内容解读

1. Consensus (共识)

- 共享内存里：只需要 Ω 就够了（最终领导者检测器）【Lynch & Hadzilacos 1994】。
- 消息传递里：需要 $\Omega + \Sigma$ (Σ 是 quorum/majority detector，保证进程能形成多数派集合)【DFG10】。

✪ 这说明在消息传递比共享内存更弱，所以要多一个 Σ 来保证多数派的交集。

2. k-set agreement

- 共享内存里：需要 k-anti- Ω （就是说系统最终会有至少 $n-k$ 个进程认同同一个领导，剩下最多 k 个可以不同）【GK09】。
- 消息传递里：？（目前没有明确答案，仍是研究中的开放问题）。

3. Set agreement

- 共享内存里：需要 anti- Ω （最终保证至少两个进程不会选同一个领导）。
- 消息传递里：需要 Σ （某种复杂的 leader/lock oracle）【DFGT08】。

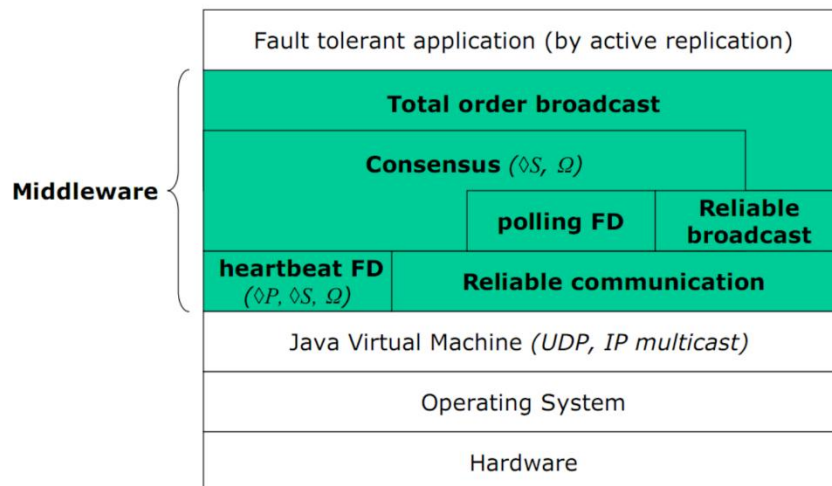
4. Eventual consistency

- 共享内存里：表格没写（可能 trivial）。
- 消息传递里：需要 Ω （最终领导者）【DKGPS15】。
- ✪ 很符合直觉：最终一致性只需要最终大家“跟随同一个 leader 的版本”就能实现。

■ 最上层: Fault tolerant application

- 这是应用层, 比如数据库、分布式存储、金融系统等。
- 通过 **active replication (主动复制)** 来实现容错: 同一份请求在多个副本上执行, 保证即使有节点挂了, 整体还能继续运行。

容错体系结构



■ Middleware (中间层)

这是整个容错架构的关键——各种容错协议和服务都在这里实现。

- Total order broadcast (全序广播)**
 - 保证所有进程接收到的消息顺序一致 (例如多副本数据库都按相同顺序应用更新)。
 - 它依赖下面的 **Consensus (共识)** 来实现。
- Consensus ($\diamond S, \Omega$)**
 - 共识需要两个故障检测器:
 - Σ (Quorum FD) 保证多数集合有交集。
 - Ω (Omega, Leader FD) 保证最终会有一个正确的领导者。
 - 两者结合才能在消息传递系统里实现共识。
- polling FD (轮询故障检测器)**
 - 一种实现故障检测的方法: 进程定期发问“你还活着吗?”
 - 用来辅助共识和可靠广播。
- Reliable broadcast (可靠广播)**
 - 保证消息只要发送了, 就要么所有人都收到, 要么都收不到 (避免分裂脑)。
 - 它依赖可靠通信和心跳 FD 来检测故障。
- heartbeat FD ($\diamond P, \diamond S, \Omega$)**
 - 基于心跳的故障检测器, 进程周期性发送“心跳”来证明自己活着。
 - 这里包含三类 FD:
 - $\diamond P$ (Eventually Perfect FD): 最终能正确判断活/挂掉。
 - $\diamond S$ (Eventually Strong FD): 最终能保证准确性。
 - Ω (Leader FD): 最终能选出唯一领导者。
- Reliable communication (可靠通信)**
 - 保证进程间的消息不会丢失/重复/篡改 (通常基于 TCP/确认重传实现)。
 - 是所有上层容错协议的基础。

■ 系统底层

- **Java Virtual Machine (UDP, IP multicast)**
 - 运行环境, 这里强调支持底层网络原语, 比如 **UDP, IP multicast (多播)**。
- **Operating System**
 - 提供进程调度、内存管理、I/O 等基本能力。
- **Hardware**
 - 最底层, 提供真实的物理运行环境。

用 P 达成共识 - 算法 1

- 基于轮次和领导者
- 进程递增执行轮次 (1 到 n)
- 在每一轮中: id 与轮数相对应的进程为领导者 (id 领导者 = id 轮数 % N)
- 领导者选择其当前值, 并决定将其分配给所有人。
- 非 "领导者 (节点 id != round id % N) 等待 :
 - (a) 收到领导者的信息, 以便选择自己的值
 - (b) 领导者的怀疑
- 在 N 轮中, 所有进程都做出了决定 (所有进程都是领导者)

用 P 达成共识: 算法 2

- f = 可容忍的最大故障数
- 每个 P_i 进程维护一个 V_i 数组, 用于存储建议值
- f+1 轮:
 - 每个 P_i 进程增量广播 V_i
 - 等待从所有非可疑进程收到向量
- f+1 轮后 :
 - P_i 选择其向量的第一个非空值做为最终决定值

f+1 轮做出决定

用 $\langle \rangle S$ 达成共识

- 旋转协调器算法 [CHT 96]

- $f < n/2$ 碰撞故障

- 进程编号为 1、2、.....、n

- 执行异步轮次

- 第 r 轮，协调器 = 进程 $(r \bmod n) + 1$

- 协调器 c :

- 强加其值 v

- 如果不怀疑 c ，则选择 v

- 每轮 4 个阶段

- 第 1 阶段：每个进程向协调器发送其当前值，并盖上最后一轮更新的印章。

- 第 2 阶段：协调者收集大多数值，估计值 = 最新值之一，分配估计值。

- 第 3 阶段：对于每个正确的进程

- 接收估计值：向协调器发送 ack 返回，更新当前值

- 怀疑协调器：发回 $nack$

- 第 4 阶段

- 协调者收到多数回应 ($ack-nack$) :

- 如果多数 ACK 回应，最终值 = 估计值 + 可靠广播值 所有进程在收到后决定收到的值

- 如果没有收到多数 ACK 回应：再来一轮

不稳定时的行为

- 检测器只需在足够长的时间内保持稳定即可（“永久”精度特性仅在算法期间有效）。

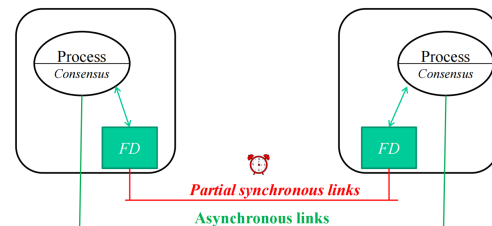
故障检测器的实施

• 指标

• 时间模型

• 实施

• 示例：缩放



这张图在告诉你：失败检测器 (FD) 是通过“部分同步模型 + 超时”来实现的，它作为额外模块，给上层的共识进程提供怀疑信号，从而绕过 FLP 不可能性。

纯异步模型 $\rightarrow$ 无法区分故障和延迟

纯异步模型 + 半同步 FD = 发现错误

时间假设 (1)

基于计时器的实现 :

- 部分同步系统

对于 $\langle \rangle P$ （最终不再出错）

- 有一个时间（GST：全局稳定时间），在这个时间内，信息传输和处理时间有一个未知的界限（M3 模型 [CHT 96]）。

=> Permet 实现 $\langle \rangle P$

想法：每次发生错误，定时器都会增加

=> 有一个时刻（GST 之后）不会再发生错误（定时器达到未知界限）。

最终完美 FD $\diamond P$

初始化: 一开始, 每个进程会怀疑其他进程的状态, 并为每个进程设置一个初始的定时器(Δ)。

任务 1: 定期发送 "心跳 (HEARTBEAT)" 消息, 告诉其他进程 "我还活着"。

任务 2: 如果在一定时间内 (Δ 时间段内) 没有收到某个进程的心跳消息, 则怀疑该进程出现故障, 将它加入 "怀疑列表"。

任务 3: 如果之后收到某个之前被怀疑的进程的心跳消息, 则将它从怀疑列表中移除, 并调整定时器 (每次延长定时器的等待时间 Δ)

时间假设 (2)

1. $\text{Pour } \langle \rangle \text{ Set } \Omega$ (最终在某一个进程上没有更多的错误):

这部分是在讨论如何在一个进程上最终不再发生错误。系统的假设被简化为一个包含最终会同步的通信通道集合 (这些通道称为 "ultimately punctual" 或 " $\langle \rangle$ -timely")。简单来说, 这些通道最终会按时传递消息, 即使在某些时候可能会有延迟。

2. p 是一个 $\diamond j$ -source:

这里定义了所谓的 " $\diamond j$ -source"。如果 p 是一个 " $\diamond j$ -source", 意味着至少有 j 条从进程 p 发出的通信链路是最终按时的 ($\diamond$ -timely)。换句话说, 进程 p 的至少 j 条链路最终会变得稳定和可预测。

注意: 我们并不知道这些链路何时会达到稳定状态 (即: 上限不明确)。

3. Ω 实现条件:

系统中的 Ω 模块可以实现, 前提是存在至少一个 " $\diamond f$ -source" 是正确的。这里 f 表示系统中的故障数量。如果有一个进程的 f 条链路最终会变得按时并且它是正确的, 那么我们可以实现 Ω 。这意味着系统能依赖这个进程来帮助其他进程达成共识或同步。

主题：如何实现 $\diamond P$ (Eventually Perfect failure detector)

• 背景

在异步系统中，消息延迟是不可预测的，失败检测器会不断误报。

在部分同步系统里，我们假设存在一个时间点 **GST (Global Stabilization Time)**，在此之后，消息传输和处理的延迟有一个固定的上界（但在 GST 之前你不知道这个上界）。

• 关键点

- GST 之后，系统表现得像同步系统 $\rightarrow$ 消息延迟 $\leq$ 某个未知常数 Δ 。
- 因此，只要不断增大超时时间 (timer)，最终能“包住”这个未知的 Δ 。
- 在此之后，失败检测器就不会再误报（即不会错把活的节点当死的）。

• 结果

$\Rightarrow$ 可以实现 $\diamond P$ (Eventually Perfect FD) :

- 一开始可能会误判（错误怀疑活节点），
- 但 **最终** 不会再犯错。

⚡ 直观理解：

“只要你不断加大超时，迟早会超过网络和处理延迟的上限，一旦超过，就不会再误报 —— 于是你得到了一个最终完美的失败检测器。”

主题：如何实现 $\diamond S$ 和 Ω (Eventually Strong FD & Eventual Leader Oracle)

• 问题

$\diamond S$ (Eventually Strong FD) 和 Ω (最终领袖检测器) 比 $\diamond P$ 的要求弱一些：

- 它们允许对某些节点永远误判，只要至少有一条“好路径”稳定存在就行。

• 假设

假设系统中存在一部分 **最终同步的通道** (ultimately timely links)，也就是说：

- 不是所有链路都可靠，但至少有一些链路最终会在有限时间内稳定传递消息。
- 这些链路的延迟上界也是未知的，但 **存在**。

• 定义

- 一个节点 p 是 $\diamond j$ -source，如果它至少有 j 条出边（发给别人的链路）是最终同步的。
- 注意：我们不需要所有链路同步，只要“部分”链路最终同步就够了。

• 结论

- 如果系统里存在至少一个 $\diamond f$ -source（即至少有 $f+1$ 条好链路，能覆盖所有非故障节点），那么我们就能实现 Ω 。
- Ω 的性质：最终大家都会同意某一个正确节点是“领导者”。

⚡ 直观理解：

“即使网络乱七八糟，只要有至少一条最终稳定的消息通路能把正确节点的信息传出去，最终大家会收敛到同一个领导者。这就是 Ω 失败检测器。”

关键点：Asynchronous implementations (sans temporisateur)

基于时间的故障检测器

意思是：不依赖“超时”的异步实现方式。

1. Query-response 模式

- 节点 p 向所有其他节点发 query，等到 **$n-f$ 个响应** 就可以继续（假设系统总共 n 个进程，最多 f 个可能失败）。
- 因为即便有 f 个挂掉，你还是能拿到 $n-f$ 个正确节点的响应。
- 这里不需要依赖时间，只是等“足够多的响应数”。

2. 已知故障上限 f

- 前提是假设你**事先知道**系统最多会有 f 个故障进程。
- 没这个信息你就不知道该等多少个回复。

3. 通信信道的相对速度假设

- 假设某些信道永远比其他的快。
- 这样可以靠“哪个先到达”来推理，而不是靠超时。
- 比如：如果你知道 $A \rightarrow B$ 的链路总是比 $A \rightarrow C$ 快，那么当 C 的消息先到，就说明 $A \rightarrow B$ 出了问题。

1. 初始化部分

- **Timeout[q]**: 为每个进程 q 设定超时时间, 初始值为 $\eta + 1$ 。
- **counter[q]**: 一个计数器, 用于记录该节点被怀疑的次数, 初始为 0。
- **suspect[q]**: 一个集合, 记录对某节点的怀疑者。
- **timer(q)**: 为每个进程 q 设置定时器, 超时时间为 **Timeout[q]**。

Ω 算法:

Suppose une $\Diamond$ **f**-source: $\exists p$ that has at least f output *eventually timely links*

on initialization:

```
1  $\forall q \neq p : \text{Timeout}[q] \leftarrow \eta + 1$ 
2  $\forall q : \text{counter}[q] \leftarrow 0, \text{suspect}[q] \leftarrow \emptyset$ 
3  $\forall q \neq p : \text{reset timer}(q) \text{ to } \text{Timeout}[q]$ 
4 start tasks 0, 1 and 2
```

Idée : un vecteur (counter) qui compte le nombre de suspicions

算法的关键思想

- **容错的检测**: 通过一个向量 **counter** 记录节点的怀疑次数, 同时使用 **Timeout** 来动态调整等待时间。
- **多进程怀疑机制**: 算法要求至少有 $n - f$ 个节点怀疑同一个节点 q 才会更新 **counter[q]**。这个机制确保单一进程的怀疑不会影响系统的正常运行, 从而提高了容错能力。

时间假设 (3)

- 异步实现 (无定时器):
- 基于查询-响应 (等待有限数量的响应 - $n - f$)
- 失败进程数量的先验知识 (f)
- 通信通道的相对假设 (比其他通道更快的通道)

2. 任务描述

算法定义了三个任务 (task 0、task 1 和 task 2), 它们不断循环执行。

Task 0: 选择领导者

- 持续循环, 每次选择一个领导者 **leader**, 满足 **counter** 数值最小的节点成为领导者。选择标准是找到 **counter** 值最小的进程 ℓ , 即 $(\text{counter}[\ell], \ell)$ 最小。

Task 1: 心跳消息广播

- 每隔一段时间 η , 发送一个 **ALIVE** 消息以及 **counter** 信息给系统中的其他进程, 用来表明自己仍然是存活的。

```
task 0:
5 repeat forever
6    $\text{leader} \leftarrow \ell \text{ such that } (\text{counter}[\ell], \ell) = \min\{(\text{counter}[q], q) : q \in \Pi\}$ 

task 1:
7 repeat forever
8   send (ALIVE, counter) to all processes except  $p$  every  $\eta$  time

task 2:
9 upon receive (ALIVE,  $c$ ) from  $q$  do
10   for each  $r \in \Pi$  do  $\text{counter}[r] \leftarrow \max\{\text{counter}[r], c[r]\}$ 
11   reset timer( $q$ ) to Timeout[ $q$ ]

12 upon expiration of timer( $q$ ) do
13    $\text{Timeout}[q] \leftarrow \text{Timeout}[q] + 1$ 
14   send (SUSPECT,  $q$ ) to all
15   reset timer( $q$ ) to Timeout[ $q$ ]

16 upon receive (SUSPECT,  $q$ ) from  $r$  do
17    $\text{suspect}[q] \leftarrow \text{suspect}[q] \cup \{r\}$ 
18   if  $|\text{suspect}[q]| \geq n - f$  then
19      $\text{suspect}[q] \leftarrow \emptyset$ 
20      $\text{counter}[q] \leftarrow \text{counter}[q] + 1$ 
```

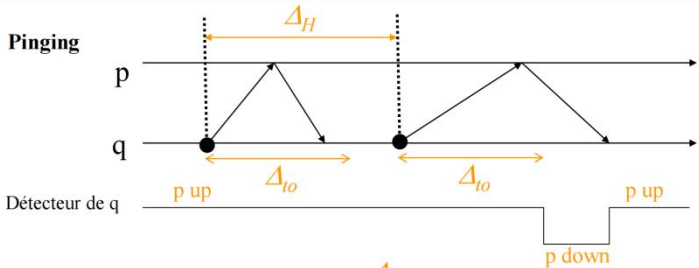
compteur de q augmenté uniquement si suspecté par au moins $n - f$ processus

Task 2: 处理接收到的消息和超时事件

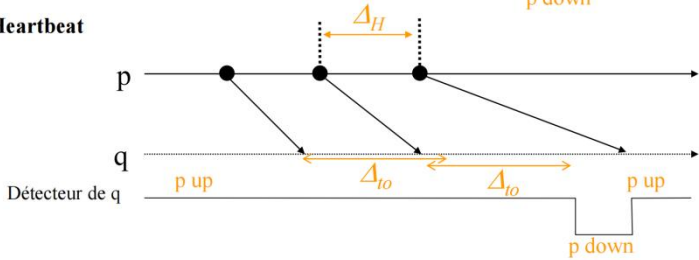
- **收到 ALIVE 消息时**:
 - 处理来自进程 q 的 **ALIVE** 消息并更新 **counter** 值: 将 **counter** 更新为当前值和 $c[r]$ 中的最大值 (取各进程最新的计数值)。
 - 重置进程 q 的 **timer** 为当前的 **Timeout**[q], 即重置超时时间。
- **超时事件发生时**:
 - 如果 **timer**(q) 超时, 表示未收到进程 q 的心跳, 则执行以下操作:
 - 增加 **Timeout**[q] 值, 以适应更长的等待时间。
 - 向所有进程发送 **SUSPECT** 消息, 怀疑进程 q 出现故障。
 - 重置 **timer**(q)。
- **收到 SUSPECT 消息时**:
 - 当从进程 r 接收到怀疑 q 的消息后, 将 r 加入到 **suspect**[q] 集合。
 - 检查 **suspect**[q] 的大小, 如果该集合的大小大于等于 $n - f$, 表示至少有 $n - f$ 个进程怀疑 q 出现问题。
 - 将 **suspect**[q] 清空并增加 **counter**[q] 值。这个计数增加表示对 q 的怀疑进一步升级, 可能接近确定其故障状态。

Techniques d'implémentation

• Ping

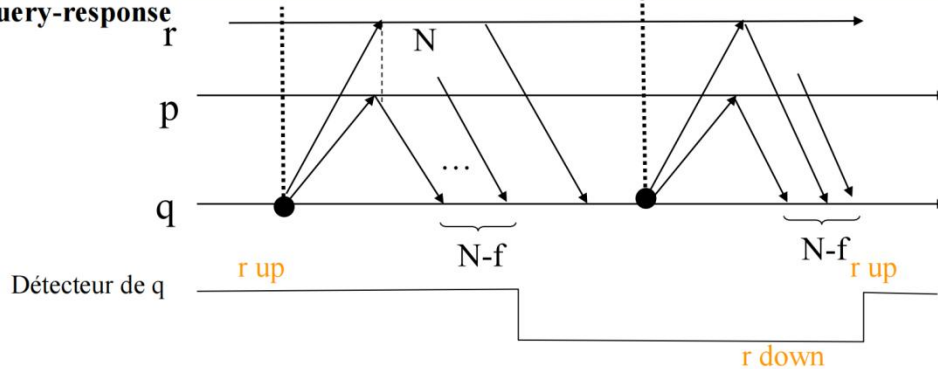


• Heartbeat



Implementations (2)

- Query-response



陈氏估计模型：（Chen et al.）

利用之前接收到的心跳消息来预测下一个心跳的到达时间，并根据这些时间点判断进程是否掉线或失效。

- Prochain temporisateur :

- Timeout (τ_{k+1}) = date (EA_{k+1}) + marge de sécurité (α_{k+1})

- Date : Chen's estimation

$$EA_{(k+1)} \approx \frac{1}{n} \left(\sum_{i=k-n}^k A_i - \Delta_i \cdot i \right) + (k+1) \cdot \Delta_i$$

- Marge : Fixe (Chen et al.)

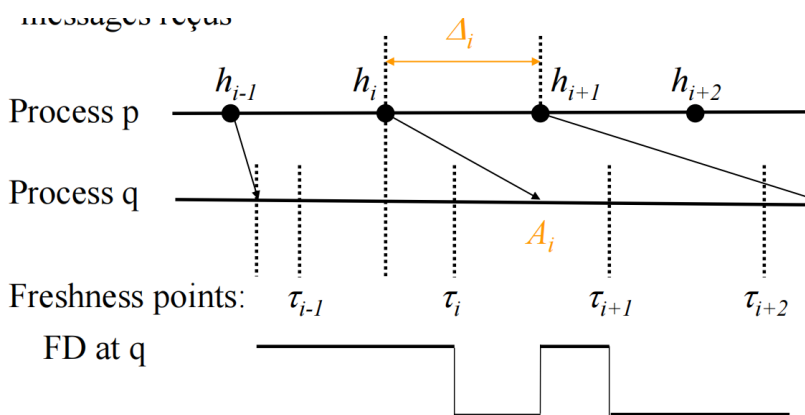
Dynamique (Bertier et al. – Basé sur RTT)

$$error_{(k)} = A_k - EA_{(k)} - delay_{(k)}$$

$$delay_{(k+1)} = delay_{(k)} + \gamma \cdot error_{(k)}$$

$$var_{(k+1)} = var_{(k)} + \gamma \cdot (|error_{(k)}| - var_{(k)})$$

$$\alpha_{(k+1)} = \beta \cdot delay_{(k+1)} + \phi \cdot var_{(k+1)}$$



1. Prochain temporisateur (下一个定时器) :

- 公式 $\text{Timeout}(\tau k+1)$ 用来估计下一个定时器的超时时间。该时间由两部分组成:
 - $\text{date}(\text{EA}(k+1))$: 表示期望收到下一个心跳消息的时间点。
 - $\text{marge de sécurité}(\alpha k+1)$: 是一个安全余量, 用来防止由于网络延迟等原因导致的误判。

2. Chen's estimation (陈氏估计) :

- 陈氏估计模型用于预测下一个心跳消息的到达时间:

$$EA(k+1) \approx \frac{1}{n} \left(\sum_{i=k-n}^k (A_i - \Delta_i \cdot i) \right) + (k+1) \cdot \Delta_i$$

- A_i : 表示第 i 次收到的心跳消息的时间。
- Δ_i : 表示进程之间发送心跳消息的间隔时间。
- n : 用于估计的过去心跳消息数量, 即使用过去的 n 次消息时间来进行预测。

3. Marge (安全余量) :

- 安全余量可以有两种方式:
 - Fixe (固定)**: 根据陈氏模型的估计, 固定地加上一些安全余量。
 - Dynamique (动态)**: 基于 Bertier et al. 提出的模型, 它使用动态计算基于 RTT (Round Trip Time, 往返时间) 的延迟和方差。

4. Bertier et al. 动态模型:

- 这是一个基于 RTT 的动态估计模型, 涉及多个公式来调整超时阈值:

- $\text{error}(k)$: 计算实际接收到的心跳消息与预测时间之间的误差。

$$\text{error}(k) = A_k - EA(k) - \text{delay}(k)$$

- $\text{delay}(k+1)$: 下一个延迟的估计值, 基于当前的延迟和误差动态调整:

$$\text{delay}(k+1) = \text{delay}(k) + \gamma \cdot \text{error}(k)$$

- $\text{var}(k+1)$: 延迟的方差, 用于估计延迟变化的波动情况:

$$\text{var}(k+1) = \text{var}(k) + \gamma \cdot (|\text{error}(k)| - \text{var}(k))$$

- $\alpha(k+1)$: 下一个安全余量, 通过当前的延迟和方差计算得出:

$$\alpha(k+1) = \beta \cdot \text{delay}(k+1) + \phi \cdot \text{var}(k+1)$$

检测质量:

1. Complétude (完整性) : 关注检测的覆盖度, 衡量探测器检测到错误的时间。

- Temps de détection (T_D)**: 检测时间, 指错误发生到被探测器检测到的时间间隔。在图中, 这个时间是在"down"状态开始后, 探测器识别到状态改变的時刻到"X"点的时间。

2. Justesse (准确性) : 关注探测器错误的频率和持续时间。

- Temps entre deux erreurs (T_{MR})**: 两次错误之间的时间间隔, 表示探测器在两次错误检测之间的稳定时间。
- Durée des erreurs (T_M)**: 错误的持续时间, 指探测器错误状态保持的时间长度。

Durée : 38 heures

Utilisation normale du laboratoire

	Historic + RTT	RTT	Historic
False detection	24	54	29
Mistake duration (ms)	31,6	25,23	36,61
Detection time (ms)	5131,7	5081,79	5672,53

- **RTT**方法在检测时间和错误持续时间上表现最好，但假检测次数最高。
- **Historic + RTT**方法在假检测和检测时间之间取得了平衡，假检测数量较少，检测时间也较短。
- **Historic**方法假检测较少，但检测时间最长且错误持续时间较长。

1. Structure en anneau (环形结构) - [WLL07] – PDC

- **环形结构** (Ring Structure) 是分布式系统中常见的一种拓扑结构，节点按照环的方式排列，每个节点连接到它的邻居节点。消息通常通过环传递，系统通过这种方式实现进程间通信。
- **PDC** 指的是并行分布式计算 (Parallel and Distributed Computing)。
- 这种结构适合大规模系统，因其简单的拓扑结构，易于实现，但在故障恢复和扩展性方面可能面临挑战。

2. Approche probabiliste (概率方法) - [GCG01] – PODC

- **概率方法** (Probabilistic Approach) 用于系统中的故障检测或消息传播等问题。它基于概率理论，允许系统以一定的概率做出决策，适用于高扩展性和不确定性的环境。
- **PODC** 是一种在分布式系统中常见的会议或研究方向，全称是 **Principles of Distributed Computing**。
- 概率方法的优势在于它不需要完全的确定性，因此在大规模和不完全同步的系统中非常有效。

3. Organisation hiérarchique (分层结构) - [BMS03] – DSN

- **分层结构** (Hierarchical Organization) 是一种通过将系统划分为多个层级（如区域、组或簇）来管理大规模系统的方法。在每一层中，节点通过层级中的领导者进行通信，降低了系统的复杂性。
- **DSN** 指的是容错计算和分布式系统的一个重要会议：Dependable Systems and Networks（可靠系统与网络）。
- 这种方法的优点在于其能提高系统的可扩展性，并在大规模系统中减少通信开销。

4. Hypercube pour HPC (用于高性能计算的超立方体) - [BBGHR16] – Supercomputing

- **超立方体结构** (Hypercube) 是一种高效的网络拓扑结构，特别适合高性能计算 (HPC) 系统。在超立方体中，节点形成类似多维立方体的结构，每个节点与多个邻居直接相连，从而减少通信延迟并提高并行计算效率。
- **Supercomputing** 指的是超级计算领域，在这里引用的可能是相关研究的会议或期刊。
- 超立方体在大规模并行计算中有很好的性能表现，适用于超级计算机等需要极高计算能力和高吞吐量的场景。

在 HPC 环境中实现应用的弹性

- 应用程序在多个节点崩溃后继续执行（最终会发生节点崩溃）
- 需要快速全面地了解组成员
- 3 个关键特性：
 1. 快速：故障检测
 2. 全面：故障知识传播
 3. 弹性机制应具有最小的影响

1. 环形结构 (Structure en anneau)

- **怎么做**：每个节点只给“下一个节点”发心跳 (heartbeat) 。
- **优点**：消息开销非常低 (每个节点只和一个邻居说话) 。
- **缺点**：检测延迟大——如果一个节点挂了，消息要沿着环慢慢传一圈，大家才知道。

👉 适合小集群或对延迟要求不高的场景。

2. 概率方法 (Approche probabiliste)

- **怎么做**：每一轮里，节点随机挑一个其他节点来监控 (发心跳) 。
- **优点**：概率上能保证最终所有节点都会被检测到，系统能扩展到很大规模 (scalability) 。
- **缺点**：超时参数难设定，而且检测的完整性和正确性只是“概率保证”，不是确定的。

👉 类似于 gossip 协议，用在超大规模系统里比较多 (像 Cassandra、Akka cluster) 。

3. 分层结构 (Organisation hiérarchique)

- **怎么做**：先在本地集群 (cluster) 里检测，选出一个代表 (proxy/leader)，再由这些代表组成一个“全局检测组”。
- **优点**：减少跨机房/跨网络的消息开销，本地检测快，全局检测也能扩展。
- **缺点**：结构更复杂，需要额外的选举和代理管理。

👉 适合 多数据中心、大规模 HPC 或云系统。

故障检测机制：

节点按照环形排列，定期向后一个节点发送心跳消息，如果后一个节点没有回应的话，则认定它失效，继续向下下个节点发送心跳消息，直到找到可以回复心跳消息的节点（即正常节点），并将所有失效节点提出系统。

Hypercube Broadcast Algorithm

寻找传递多个广播消息副本的不相交路径

不相交路径（Disjoint Paths）

是指在图论或网络通信中，两条路径之间没有共同的节点或边。

不可靠检测器的优势：

- 将故障检测作为一种抽象：允许从同步性中抽象出来
- 快速故障检测（不可靠方面）
- 可绕过 FLP

检测器的缺点：

- 假定信道可靠 => 有些检测器假定“公平有损信道”。
- 不现实的永久正确性属性 => 永久性仅限于算法的持续时间
- 简单故障模型（崩溃）：扩展至遗漏故障

- **不可靠故障检测器的好处：**

- 提供抽象，简化共识算法的设计。
- 快速发现问题。
- 能绕过 FLP 不可能性。

- **不足：**

- 假设过于理想（信道可靠/公平）。
- 永久正确性在现实系统里很难保证。
- 故障模型太简单（只考虑 crash，而不考虑遗漏、拜占庭等更复杂情况）。

Paxos 是一种共识算法

轮值协调员方法的局限性

- 需要可靠的信道（不现实）
- 信息丢失的影响：阻塞

关于轮次与协调：

需要避免在某一轮次被卡住。

通过定时器的超时来改变轮次，或者当节点变化时改变轮次。

维护 Leader 的概念来保持多数派了解当前值。

Paxos 假设：

通信方面：

异步通信，消息不会被篡改，但有丢失消息的可能。

进程方面：

进程数量固定。

容许“crash-recovery”（即节点故障后可以恢复）的情形，每个进程拥有持久化状态。

✦ Paxos 是什么

Paxos 是 **分布式一致性算法 (distributed consensus algorithm)**，由 Leslie Lamport 在 1990 年代提出。

它的核心目标是：

在一个 **可能会有故障**（节点宕机、消息丢失、网络延迟）的分布式系统里，**让多个节点对某个值达成一致**。

✦ 为什么需要 Paxos

想象一下：

- 有一群服务器需要共同决定“某个日志条目”写入什么内容（例如 Raft 里的 replicated log）。
- 网络里有延迟、丢包，某些服务器可能挂掉或重启。
- 但系统必须保证：
 1. **安全性 (safety)**：所有幸存节点最终的决定必须一致，不能出现分叉。
 2. **活性 (liveness)**：如果多数节点正常，系统总能继续前进。

Paxos 就是为了解决这个问题而提出的。

✦ Paxos 的核心思想

1. 多数派 (majority quorum)

- 只要一个值被大多数节点接受，它就“安全”，不会被其他值覆盖。
- 因为任何两个多数派必定有交集，交集里的节点会“记住”已有的决定。

2. 两阶段协议 (two-phase protocol)

- **阶段 1 (Prepare/Promise)**：提议者 (Proposer) 提出一个编号较大的提案，要求接受者 (Acceptor) 承诺“不会再接受更小编号的提案”。
- **阶段 2 (Accept/Accepted)**：如果提议者拿到了大多数 Acceptor 的承诺，就可以发出“接受这个值”的请求，大多数 Acceptor 接受后，该值就被选定 (chosen)。

3. 容错性

- 只要 **超过半数的节点存活**，Paxos 就能继续达成一致。

✦ Paxos 的优点与缺点

• ✓ 优点

- 保证安全性：不会产生冲突的决定。
- 只要多数节点存活，就能继续工作。

• ✗ 缺点

- 算法比较复杂、难以实现 (Lamport 的论文最初写得很绕)。
- 通信开销较大 (至少两轮)。
- 实现上的工程复杂度较高，所以后来出现了 **Raft**，它和 Paxos 原理接近，但更容易理解和实现。

1. 基于 Leader 的机制：

Paxos 依赖一个“Leader”节点来主导共识过程，具体如下：

- **Leader 通过探测器 Ω 确定**：通常，系统中会有一个机制来检测哪个节点可以作为 Leader，确保一个节点作为 Leader 发起提案。
- **Leader 启动一个新的“ballot”**：Leader 负责开始一个新的投票轮次（即轮次、视图、表决），并尝试获得多数节点的同意。
- **目标是争取多数节点的加入**：Leader 在提出提案时，会寻求得到系统中多数节点的响应和认可，以达成共识。

2. 节点总是加入最近的“ballot”：

节点（参与者）会选择跟随最新的投票轮次，忽略旧的提案。这是为了避免对旧的、可能已经无效的提案进行投票，从而确保整个系统向前推进。

3. Paxos 的两个阶段：

Paxos 算法分为两个主要阶段：

- **阶段1：收集以前轮次的投票结果**：Leader 先从多数节点那里收集以前轮次的投票结果，看看是否已经有某个提案获得了多数认可。如果是，它会继续基于这个提案工作。
- **阶段2：提出新的提案并寻求批准**：如果没有获得明确的提案，Leader 会提出一个新的提案，并尝试获得多数节点的同意。

4. 算法停止的条件：

当某个唯一的 Leader 成功完成 2 轮次的通信，并得到系统中多数节点的认可时，算法会停止。这意味着共识已经达成。

5. 备注：

- **可能存在多个竞争的 Leader**：在 Paxos 中，不同的节点都可以成为 Leader 并同时提出提案。这种竞争通过 ballot 编号机制进行区分和处理。
- **Ballot 编号用于区分不同 Leader 提出的提案**：每个 Leader 发起的提案会有不同的 ballot 编号，这个编号可以帮助节点区分不同的提案，从而选择最新的提案来继续处理。

- Chaque valeur est associée à un **numéro de ballot**
- Numéro de ballots avec un ordre total
- Pairs $\langle \text{num}, \text{process id} \rangle$
- $\langle n_1, p_1 \rangle > \langle n_2, p_2 \rangle$
 - Si $n_1 > n_2$
 - ou $n_1 = n_2$ et $p_1 > p_2$
- Le leader courant p choisit localement un numéro unique croissant :
 - Si le dernier ballot connu est $\langle n, q \rangle$ alors p choisit $\langle n+1, p \rangle$

图片 1: Paxos 的 Ballot 编号

1. 每个值都与一个 Ballot 编号相关联:

- 在 Paxos 算法中, 每一个被提议的值都会与一个唯一的 Ballot 编号相关联, 用于区分不同提案的顺序。

2. Ballot 编号有一个总的顺序:

- Paxos 使用一个全局顺序 (total order) 的编号机制来确保不同的提案能够在分布式系统中有序进行。

3. Ballot 编号的形式为 $\langle \text{num}, \text{process id} \rangle$:

- 每个 Ballot 编号都是一个二元组, 包含了 Ballot 数字 (num) 和进程 ID (process id)。其中 num 用于表示 Ballot 编号, process id 用于区分由哪个进程发起。

4. 编号比较规则:

- 比较两个 Ballot 编号 ($\langle n_1, p_1 \rangle$ 和 $\langle n_2, p_2 \rangle$) 时, 遵循以下规则:
 - 如果 $n_1 > n_2$, 那么 $\langle n_1, p_1 \rangle$ 大于 $\langle n_2, p_2 \rangle$ 。
 - 如果 $n_1 = n_2$, 则比较进程 ID, $p_1 > p_2$ 则认为 $\langle n_1, p_1 \rangle$ 大于 $\langle n_2, p_2 \rangle$ 。

5. Leader 选择一个唯一递增的 Ballot 编号:

- 当前的 Leader 通过选择一个唯一的递增 Ballot 编号来发起新的提案。如果上一个已知的 Ballot 是 $\langle n, q \rangle$, 那么 Leader 会选择 $\langle n+1, p \rangle$ 来继续提案。

图片 2: Paxos 的本地变量

1. Val_i:

- 表示当前的值 (当前的提案值)。

2. BallotNum_i:

- 表示节点 p_i 参与的最后一个 Ballot 编号, 初始值为 $\langle 0, 0 \rangle$, 对应的是第一阶段 (Prepare 阶段) 的参与情况。

3. AcceptNum_i:

- 表示节点 p_i 在第二阶段 (Accept 阶段) 接受的 Ballot 编号, 初始值为 $\langle 0, 0 \rangle$ 。这个变量用于存储节点接受的最新提案。

4. AcceptVal_i:

- 表示节点 p_i 在第二阶段 (Accept 阶段) 接受的最新值, 初始值为 $\perp$ (表示没有接受任何值)。这个变量存储了节点在共识过程中最终接受的提案值。

Paxos : Structure – Les variables locales

- Val_i
Valeur courante
- BallotNum_i, initialement $\langle 0, 0 \rangle$
Numéro du dernier ballot auquel p_i a pris part (phase 1)
- AcceptNum_i, initialement $\langle 0, 0 \rangle$
Numéro du ballot associé à la dernière valeur acceptée par p_i (phase 2)
- AcceptVal_i, initialement $\perp$
Dernière valeur acceptée (phase 2)

1. Phase 1: 准备阶段 (Prepare Phase)

- **目标:** 在这个阶段, Leader 会尝试发起一个新的投票轮次 (ballot), 并收集以前投票的决策信息。

- **步骤:**

1. Leader 发起新的 ballot:

- 定期由 `p_i` 执行, 如果 `p_i` 是 Leader, 则它会创建一个新的 Ballot 编号 `<BallotNum_i.num+1, p_i>`。
- 然后它发送 `prepare` 消息和 Ballot 编号给所有节点。

2. 节点接收 prepare 消息:

- 当节点 `p_j` 接收到 `p_i` 发来的 `prepare` 消息时, 如果接收到的 ballot 编号 `bal` 大于或等于它当前的 `BallotNum`, 则它更新 `BallotNum` 为 `bal`, 并回复 `ack` 消息, 同时包含它当前存储的 `AcceptNum_j` 和 `AcceptVal_j`。

2. Phase 2: 接受阶段 (Accept Phase)

- **目标:** 在准备阶段得到大多数节点的确认后, Leader 会提出一个新的值, 并尝试得到节点的接受。

- **步骤:**

1. Leader 收到大多数 ack 消息:

- 如果 `p_i` 从 $n-f$ 个进程中收到了 `ack`, 它会检查这些 `ack` 中的值。
- 如果所有节点的值都是 `⊥`, 则 Leader 提出一个初始值。如果不是 `⊥`, 则 Leader 会选择与最大 `b` 相关联的值作为提案值。
- 然后, Leader 会向所有节点发送 `accept` 消息, 包含 Ballot 编号和提案值。

2. 节点接收 accept 消息:

- 当 `p_j` 接收到 `accept` 消息时, 如果接收的 Ballot 编号大于等于它当前的 `BallotNum`, 则节点更新其 `AcceptNum_j` 为 Ballot 编号, `AcceptVal_j` 为接收到的值, 并向其他节点广播 `accept` 消息。

3. Phase 3: 决策阶段 (Decision Phase)

- **目标:** 在大多数节点接受了提案值后, 系统进入决策阶段, 最终确定共识值。

- **步骤:**

1. 节点接收多数的 accept 消息:

- 一旦某个节点 `p_i` 从多数节点 ($n-f$ 个) 接收到 `accept` 消息, 它决定值 `v` 为提案值 `v`, 并周期性地向所有节点广播 `decide` 消息, 告诉它们最终的共识值。

2. 节点接收决策消息:

- 当其他节点接收到 `decide` 消息时, 它们会将 `v` 作为最终的决定值并作出决策。
- **消息丢失的处理:** 如果某个值迟迟没有达成共识, 可能是因为消息丢失, 此时可以通过更改 Ballot 来启动新一轮投票, 以恢复进程。

第一阶段：准备 (Prepare)

1. Leader 出马

假设某个节点 p_i 觉得自己要当领导了，它会生成一个比之前都大的编号 (`BallotNum`)，相当于举个手说：

👉 “大家注意，我要开始一个新一轮投票啦！”

然后它把 **prepare 请求** 发给所有节点。

2. 大家表态

其他节点 p_j 收到后会检查：

- 如果这个编号 `bal` 比它见过的都大，就答应：

👉 “好，我以后不答应比你编号小的啦。”

然后它把自己以前投过的提案（如果有的话）告诉 leader（就是 `AcceptNum`，`AcceptVal`）。

第二阶段：接受 (Accept)

1. Leader 决定提案值

Leader 收到足够多（大多数）节点的回复：

- 如果大家之前都没投过，那它可以提出一个自己的新值。
- 如果有人之前投过了，那它必须沿用 **编号最大** 的那个值。
(保证不会出现两个不同的值都被选中！)

然后 Leader 广播一个 **accept 请求**：

👉 “大家，请接受这个值 v ，编号是 b ！”

2. 大家再次表态

如果节点 p_j 看到这个请求的编号 $\geq$ 自己最新承诺的编号，就会答应：

👉 “我接受这个值 v 。”

并且记住它。

第三阶段：决策 (Decide)

1. Leader 宣布结果

一旦 Leader 收到大多数节点的“接受 (accept)”回复，它就认为这个值 **被选定 (chosen)**。

然后广播：

👉 “最终决定了！大家都用值 v ！”

2. 大家定案

所有节点收到后，就确定了结果。

Paxos 至此完成一次共识。

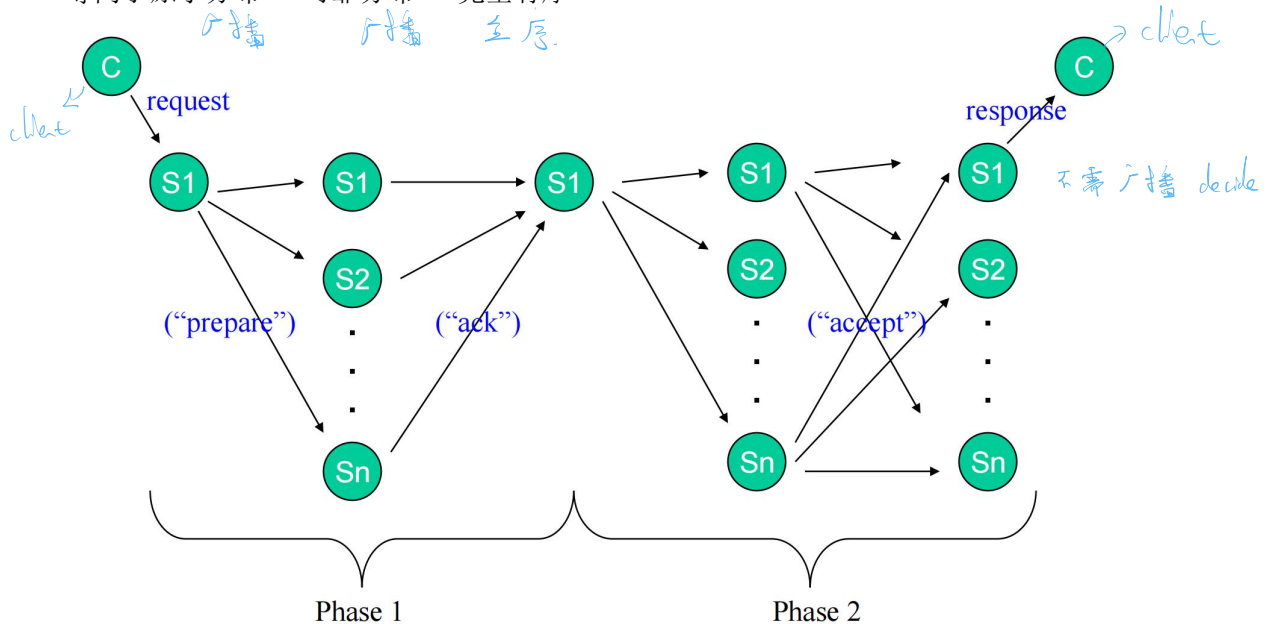
Paxos 的主要特点

- 弱假设（异步、信息丢失、崩溃恢复）
- 实际意义
- 可能的优化
 - 进程 1 可直接提出其值
- 不保证终止（FLP 始终有效），除非信道最终可靠

将 Paxos 用于复制状态机

→ 有序

- SMR: 状态机复制
- 在 n 台服务器上复制数据
- 客户端生成操作
- 所有正确的服务器必须以相同的顺序执行操作
- 就操作顺序达成一致
- 等同于原子分布 = 可靠分布 + 完全有序



Paxos 造面已状态机复制:

改进 Paxos 算法以适配并发:-

1. 多个并行的共识实例（每个请求一个）=> 每个客户端增加一个请求编号 (ReqNum)

- 解释: 为了能够处理多个并行请求, 每个请求需要分配一个唯一的请求编号 ReqNum。这样可以确保每个请求在共识过程中独立处理, 并避免冲突。

2. 将局部变量转换为数组 (无边界)

- `AcceptNum[r], AcceptVal[r], r = 1, 2, ...`
- 解释: 将算法中的局部变量 (例如 `AcceptNum` 和 `AcceptVal`) 扩展为数组, 以便存储不同请求的状态。这种转变允许每个请求都有独立的共识状态, 并且支持无限多个请求 (理论上)。 `r` 表示不同的请求编号。

1. Hypothèse faible

弱假设 (asynchrone, perte de message, crash-recovery)

- Paxos 不需要强假设, 它在异步网络下也能工作:
 - 消息可能延迟、丢失、乱序;
 - 节点可能崩溃, 然后再恢复 (crash-recovery)。
- 这意味着 Paxos 鲁棒性很强, 即使在最恶劣的分布式环境下也能保证一致性。

2. Intérêt pratique

实用价值

- Paxos 虽然理论复杂, 但在实际系统里非常有价值。
 - 比如: Google Chubby, ZooKeeper, etcd, Consul 都基于 Paxos 或 Raft。

3. Optimisations possibles

可以优化

- 举例: Le processus 1 peut directement proposer sa valeur
 - 这里指的是快速 Paxos 或者一种优化策略:
 - 如果一开始有一个“大家认可的 leader”, 它可以直接发出提案, 而不用走完整的 Paxos 准备阶段。
 - 这样就减少了通信轮数, 提高性能。

4. Terminaison non assurée

不能保证终止 (FLP 定理适用)

- FLP 不可能性结果 (Fischer-Lynch-Paterson theorem):
 - 在一个完全异步的系统里 (网络延迟无限, 节点可能随时挂), 没有算法能保证一定达成共识。
- Paxos 也逃不过这个定理 → 它只能保证安全性 (不会出错), 不能保证活性 (一定会结束)。
- 但如果假设通信通道最终是可靠的 (messages 最终一定送达), 那么 Paxos 在实践里是可以结束的。

1. 多个共识实例并行

- 每个请求 (requête) 需要一个共识实例。
 - 所以要给每个请求加一个 请求编号 ReqNum (由客户端分配)。
👉 想象一下：每条客户端请求都要跑一遍 Paxos，为了区分它们，就必须给每条请求编号。
-

2. 局部变量变成数组

- 原本 Paxos 里，每个节点只需要记住：
 - `AcceptNum` (它承诺过的最大编号)
 - `AcceptVal` (它接受过的值)
 - 但现在要同时跑很多个共识，就需要把这些变量变成“数组”：
 - `AcceptNum[r]`, `AcceptVal[r]` (r 表示第 r 个请求)。
- 👉 也就是说，不再是“一个共识的记忆”，而是“为每个请求单独存一份”。

3. 消息里加请求号

- 所有 Paxos 的消息 (prepare、accept 等) 里，都要带上请求编号。
👉 不然大家会搞混：这条消息到底是第 1 个请求的，还是第 2 个请求的？
-

4. 状态机执行顺序

- 状态机必须按请求编号的顺序来执行：
 - 先执行 `AcceptVal[1]`
 - 再执行 `AcceptVal[2]`
 - ...依次类推。
 - 每个请求在达成共识之后，leader 才能把结果返回给客户端。
- 👉 人话版：就像排队点菜，每个 Paxos 实例是一次点单，必须按顺序上菜，不能乱。

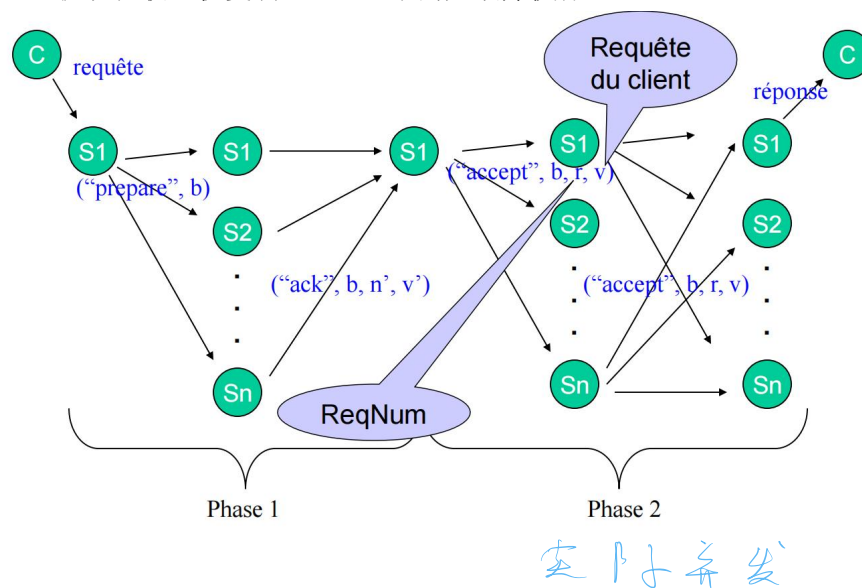
3. 在接受消息中添加请求编号

- 解释：在每个“接受”消息中添加请求编号，以确保消息能够与具体的请求实例关联。这有助于每个请求的状态独立处理，防止消息混淆。

4. 在状态机上的操作顺序

- AcceptVal[1]，然后是 AcceptVal[2]，等等
- 解释：状态机按照请求的顺序依次执行操作。例如，首先处理 AcceptVal[1]，接着是 AcceptVal[2]，如此类推。这样做的目的是保持请求的顺序性，以确保最终状态的一致性。
- 共识达成后，由领导者 (leader) 向客户端发送响应
- 解释：一旦每个请求达成共识，领导者负责向客户端返回该请求的响应。这意味着只有领导者才会回复客户端，避免多重回复的混乱情况。

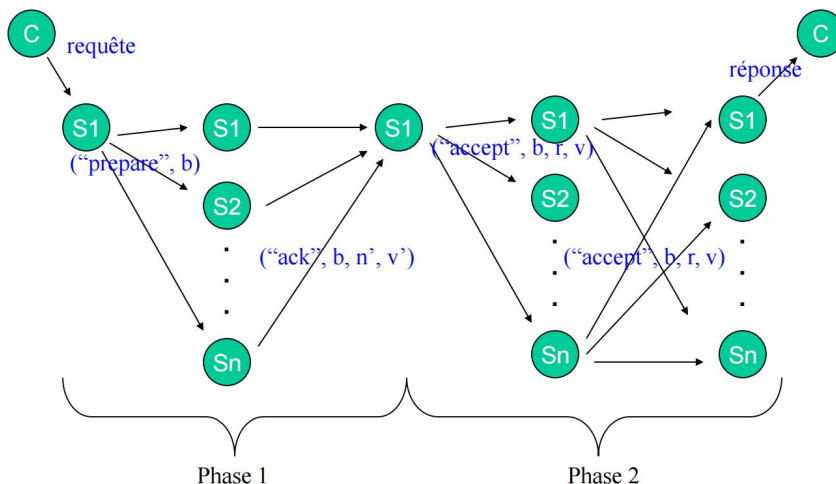
Paxos 协议在状态机复制 (SMR) 中的无故障执行



并行执行

优化第一阶段：

使得每次请求的准备工作提前完成，减少了在后续请求中的准备时间



普通 Paxos 的问题

- 标准 Paxos 每次共识要跑两轮：
 1. Phase 1: proposer (发起者) 先发 `prepare(b)` → acceptor 回复 `ack`。
 2. Phase 2: proposer 再发 `accept(b, v)` → acceptor 接受 → 达成共识。
 - 这样一个请求下来至少要 **两次网络往返**，延迟比较高。
-

优化思路：Phase 1 提前做

- 既然 Phase 1 的作用就是 **抢占 ballot 编号并获得承诺**，那能不能提前把这一步做掉？
- 办法：
 - 选出一个“稳定的 leader”（例如 S1）。
 - 让 leader 在一开始就把 Phase 1（`prepare`）广播给大家。
 - 一旦大多数 acceptor 都确认过这个 leader 的编号，就不用对每个请求都重复 Phase 1。

图里的流程

1. Phase 1 (提前准备阶段)

- Leader (S1) 向所有服务器 S2...Sn 发 `("prepare", b)`。
- 他们返回 `("ack", b, n', v')`，意思是：确认了编号 b，并告诉 leader 之前承诺过的最大编号 n' 和对应值 v'。
- 一旦 leader 得到大多数 ack，就锁定了 ballot b 的合法性。

2. Phase 2 (请求真正到来时)

- 当客户端 C 发来请求 (requête)，S1 直接用之前确认过的 ballot b 来发 `("accept", b, r, v)`。
 - 各个服务器收到后，因为已经认同了 b，不用再跑 Phase 1，直接进入 Phase 2。
 - 多数节点确认 `accept` 之后，S1 就可以把结果 (réponse) 返回给客户端。
-

优化效果

- 标准 Paxos：每次请求要 **Phase 1 + Phase 2**（两次网络 RTT）。
- 优化 Paxos：Phase 1 只跑一次（leader 选定时），之后每个请求只需要 **Phase 2**（一次 RTT）。
- 延迟从 **两跳 → 一跳**，吞吐量提升明显。

基于 Paxos 的状态机复制 (SMR) 具体实现

1. 接收 ("request", v) 消息:

- 如果当前节点不是领导者, 则将请求转发给领导者。
- 如果是领导者, 则为请求生成新的请求编号 `ReqNum`。
- 发送 "accept" 消息: 领导者向所有节点发送 ("accept", `BallotNum`, `ReqNum`, v) 消息, 其中 `BallotNum` 是提案编号, `ReqNum` 是请求编号, v 是提案值。

2. 接收 ("accept", b, r, v) 消息:

- 接受提案: 节点记录提案编号 b 和对应的值 v, 并更新 `AcceptNum[r]` 和 `AcceptVal[r]`。
- 广播 "accept" 消息: 接受提案后, 节点向所有其他节点发送确认 ("accept", b, r, v)。

拜占庭容错的安全背景

- 数据安全 —— 加密

机密性

完整性

真实性

- 处理安全性 —— 复制和任意容错算法

各种故障和错误

恶意攻击

拜占庭 Paxos (Byzantine Paxos, BFT Paxos)

它就是把 Paxos 的思想扩展到拜占庭容错场景。

1. 目标

- 让集群在有恶意节点存在时, 依然能对某个值达成一致。
- 容忍 f 个拜占庭节点, 需要至少 $3f+1$ 个节点 (这是拜占庭容错的经典条件)。

2. 基本思路

- 在 Paxos 的 prepare / accept 流程基础上, 加入了 签名 / 投票证明, 避免节点撒谎。
- 每条消息都需要加密签名, 别的节点才能验证真伪。
- 通过“多数 + 加密证明”, 保证即使有恶意节点, 也不能同时让两组 honest 节点接受两个不同的值。

3. 工作流程 (简化)

- Phase 1: Proposer 广播提案 (带签名)。
- Phase 2: Acceptor 广播投票 (附带签名)。
- 当收集到 $2f+1$ 份有效签名 (多数 honest + 可能少数恶意) 时, 值被确认。
- 其他节点可以通过验证签名来确认这是合法的共识结果。

4. 代表协议

- PBFT (Practical Byzantine Fault Tolerance) → 拜占庭 Paxos 的一个高效工程化版本。
- 被很多区块链 (如 Tendermint, Hyperledger) 采用。

拜占庭 Paxos



f: 拜占庭错误的服务器数量

为了容忍 f 个拜占庭故障节点, 系统中的总节点数 n 必须满足 $n \geq 3f + 1$ (普通 Paxos 则是 $2f+1$)。客户端接收到的 n - f 个响应中至少有 n - 2f 是来自非拜占庭的服务器, 才能确保获得正确的响应。

Paxos Byzantin : Modèle

图片 1: 拜占庭 Paxos 模型

1. 进程数量 n:

- 系统中存在 n 个进程 (从 1 到 n), 这些进程可能会出现拜占庭故障 (即恶意或不可靠的行为)。

2. 拜占庭故障的最大数量 f:

- 在系统中, 最多允许 f 个拜占庭故障, 并且 f 必须小于 $n/3$ 。这是一个核心约束条件, 用来确保系统在有拜占庭节点的情况下仍然能达成共识。
- 为了简化计算, 系统中节点总数满足 $n = 3f + 1$ 。

3. 公钥基础设施 (PKI) 认证:

- 使用公钥基础设施 (PKI) 进行身份认证。PKI 通过加密技术确保消息传递过程中节点身份的真实性, 防止恶意节点伪造身份。

4. 可靠的信道:

- 系统中的通信信道是可靠的, 但存在“诚实”故障 (即节点可能会崩溃), 并且这些故障不会自动恢复。

- n processus: $\{1, \dots, n\}$
- f fautes byzantines, $f < n/3$
 - Pour simplifier $n = 3f + 1$
- Authentication par clé publique (PKI)
- Canaux fiables, faute franche sans recouvrement

1. Le Leader peut choisir une valeur différente que la plus grande acceptée par les $n-f$ processus
 - **Solution** : Le leader doit prouver qu'il ne ment pas en envoyant les messages "ack" reçus à tous les processus
2. Si aucune valeur n'a été acceptée, le Leader peut envoyer une nouvelle valeur différente à chacun des processus
 - **Solution** : Avant d'accepter une valeur proposée par le Leader, un noeud vérifie que la valeur a été proposée à assez de processus => une phase supplémentaire (**Phase Propose**)
3. Les agents peuvent envoyer dans la phase 2 des "accept" non valides
 - **Solution** : attendre $n-f-1$ "accept" messages
4. Les agents peuvent envoyer de valeurs plus grandes dans les "ack"
 - **Solution** : Ajouter les messages "propose" signés dans les "ack" (ensemble "Proof")

图片 2: 为确保安全性的经典 Paxos 拓展

安全性意味着即使在有拜占庭节点的情况下，系统也要确保不接受错误的提案。以下是 Paxos 在拜占庭环境下的一些改进：

1. **Leader 选择不同的值：**
 - **问题：**Leader 可能选择与其他 $n - f$ 个进程接受的最大值不同的值。
 - **解决方案：**Leader 必须向所有进程展示它收到了的"ack"消息，证明它没有撒谎。
2. **没有被接受的值时：**
 - **问题：**如果没有任何值被接受，Leader 可以发送一个新的不同的值。
 - **解决方案：**在接受新的值之前，每个节点要验证该值已被足够多的节点提议。这需要添加一个新的阶段（称为"**Phase Propose**"），在该阶段验证值的正确性。
3. **无效的"accept"消息：**
 - **问题：**在第二阶段，恶意节点可能发送无效的"accept"消息。
 - **解决方案：**节点应等待 $n-f = 2f + 1$ 个"accept"消息，确保足够多的节点同意提案。
4. **发送更大的值：**
 - **问题：**节点可能在"ack"消息中发送一个过大的提案编号。
 - **解决方案：**节点必须将签名过的提案编号（"Proof"）作为证据，附加在"ack"消息中。

图片 3: 为确保活跃性的经典 Paxos 拓展

活跃性指的是系统能够继续前进，而不会因为某些节点的问题陷入死锁或活锁。以下是一些活跃性问题及解决方案：

1. **Leader 可能阻塞算法（死锁问题）：**
 - **问题：**如果当前的 Leader 出现问题（例如崩溃或不响应），整个算法可能会陷入死锁。
 - **解决方案：**引入轮替 Leader 的机制，当 Leader 无法工作时可以选举新的 Leader。此机制使用循环轮替的协调器，即 $\text{BallotNum} \bmod n + 1$ 。
2. **拜占庭进程引发的活锁问题：**
 - **问题：**拜占庭进程可能会不断地改变 Leader，导致系统无法达成共识，形成活锁。
 - **解决方案：**只有当 $f+1$ 个进程提议更换 Leader 时，才接受新的提案编号，从而避免拜占庭节点频繁干扰选举过程。

1. Le leader peut bloquer l'algorithme (deadlock)
 - **Solution** : Proposer un nouveau Leader quand il ne répond pas
 - Utiliser un coordinateur tournant $(\text{BallotNum} \bmod n) + 1$
2. Des processus byzantins peuvent changer en permanence de leader (livelock)
 - **Solution** : Accepter un nouveau "ballot" seulement si $f+1$ processus proposent un nouveau leader (=> diffusion par tous du "prepare")

初始参数：

Int	BallotNum,	initialement 0
Int	PropNum,	initialement 0
Int	AcceptNum,	initialement 0
$\text{Value} \cup \{\perp\}$	AcceptVal,	initialement $\perp$
$\text{Value} \cup \{\perp\}$	Val,	initialement $\perp$
Message Set	Proof,	initialement vide

$\text{Leader} = (\text{BallotNum} \bmod n) + 1$

Phase 1: Prepare (准备阶段)

1. Leader 计时器到期:

- 当 Leader 的计时器到期, 它会增加 Ballot 编号并向所有进程发送 prepare 消息。消息内容包含新的 Ballot 编号 (BallotNum)。

2. 接收到 prepare 消息:

- 当某个进程收到 Leader 的 prepare 消息时, 它会检查收到的 Ballot 编号 b 与自己当前的 Ballot 编号 BallotNum 的大小:
 - 如果 b 小于当前的 BallotNum, 说明这是一个过期的提案, 忽略此消息。
 - 如果 b 大于当前的 BallotNum, 那么更新 BallotNum 为 b 并向所有进程发送新的 prepare 消息。
- 最后, 进程将 ack 消息返回给 Leader, ack 消息中包含 BallotNum、AcceptNum、AcceptVal 以及证明 (Proof)。

Phase 2: Propose (提案阶段)

1. Leader 收到 ack 消息:

- Leader 从 $n-f$ 个节点收到签名的 ack 消息, 并将这些消息保存到集合 s 中。
- Leader 检查集合 s 中的有效值:
 - 如果所有有效的 vals 都是 $\perp$, 则 Leader 提出一个初始值作为提案。
 - 否则, Leader 选择集合中与最大 b 关联的有效值作为提案值。
- 然后, Leader 向所有进程发送 propose 消息, 消息包含提案的 BallotNum、提议的值 val 以及收到的签名消息集合 s 。

2. 接收到 propose 消息:

- 当节点接收到 propose 消息时, 它会进行两步验证:
 - 如果收到的 BallotNum 小于节点当前的 PropNum, 则忽略该提案, 因为该提案是旧的。
 - 如果提案值 v 不在集合 s 中的有效值范围内, 忽略该提案。
- 如果验证通过, 节点更新 PropNum 为 BallotNum 并将 propose 消息发送给所有进程。

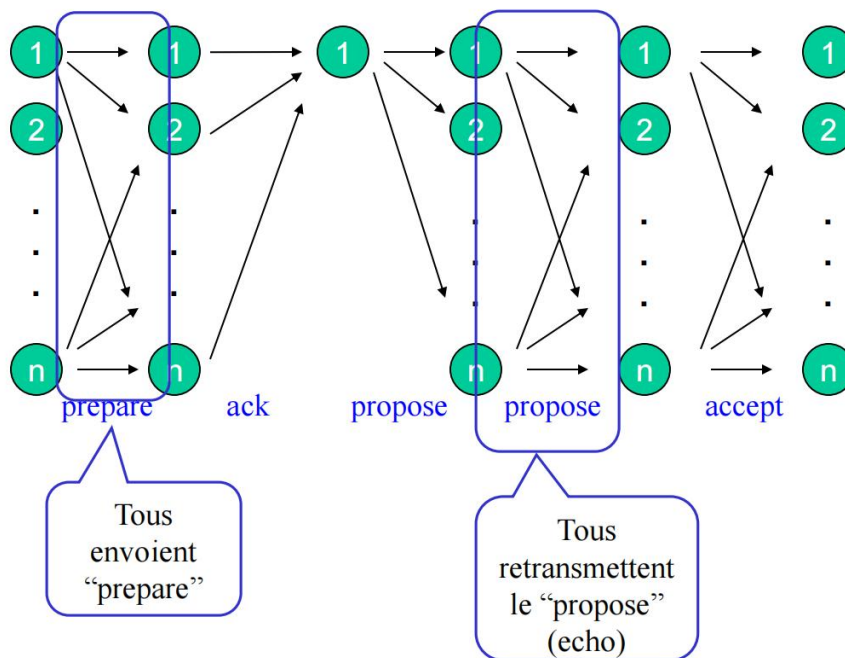
Phase 3: Accept (接受阶段)

1. 接收到 propose 消息:

- 节点收到 propose 消息后, 会检查提案的 Ballot 编号 b :
 - 如果 b 小于当前的 BallotNum, 则忽略该提案。
 - 否则, 节点更新自己的 AcceptNum 为 b , 并将提议的值 v 设置为 AcceptVal。
- 节点将 accept 消息发送给所有其他节点, 消息中包含 b 、 v 以及签名的 propose 消息集合 (Proof)。

2. 接收到 accept 消息:

- 当节点从 $n-f$ 个节点接收到 accept 消息时, 意味着系统中多数节点同意了提案, 节点就可以决定采用值 v 。



Paxos 结论 (经典)

- 容忍信息丢失的共识
- 可容忍坦诚故障和瞬时故障 (恢复)
- 高效:
 - 故障协调器之间无需轮换
 - 用于保持副本之间的一致性

1. Consensus tolérant les pertes de messages

👉 Paxos 能容忍消息丢失。

也就是说，即使网络里有消息丢了（比如丢包），只要不断重试，最终还是能达成共识。

2. Tolère les fautes franche et transitoire (recovery)

👉 Paxos 能容忍故障，包括：

- **永久性故障 (crash faults)**：节点宕机，之后可能一直不回来。
- **暂时性故障 (transient faults)**：节点挂了又恢复 (recovery)。

注意：这里讲的是 **Crash Fault Tolerance (CFT)**，不是拜占庭。也就是说 Paxos 假设节点最多就是宕机，不会乱发坏消息。

3. Performant (性能方面)

- **Pas de rotation entre les coordinateurs défaillants**

👉 Paxos 里没有频繁的 leader 选举/轮换机制。

它的 proposer（提议者）机制避免了每次都要重新找 leader，这样效率高一些。

- **Utiliser pour maintenir la cohérence entre les copies**

👉 Paxos 用来保证副本之间的一致性。

在分布式系统里，每个节点都有一份复制的数据，Paxos 就是用来保证它们“写操作的顺序一致”。